Network Management and Monitoring

AI5049, Winter Semester 2026/27

12 October 2026

Lucas Immanuel Nickel

About me

Lucas Immanuel Nickel

- Network Operations Engineer at DE-CIX
- Research Associate at Fulda University of Applied Sciences
- M.Sc. Applied Computer Science, research stay at the University of Toronto

What is this course about?

Find faults. Make safe changes. Measure the service.

Use that experience to investigate one network-management question.

Our lens: large IP networks

ISP backbones, transit, peering and IXPs.

We study how operators keep these networks working.

The technical block

Date	Time	Topic
12 October 2026	13:45–17:00	Networking 101 + Operations
13 October 2026	08:45–15:15	Automation + Incidents
14 October 2026	08:45–15:15	Monitoring / Observability

Short demos, then time to try it yourself.
Use the university lab PCs with Containerlab preinstalled.

ReefNet, from four sides

1. **Understand**
   Healthy request
2. **Operate**
   One incident
3. **Automate**
   Safe repetition
4. **Observe**
   Earlier evidence

The project workshops

Friday sessions: **13:45–18:45**

Date	Focus
30 October 2026	Research questions and sources
13 November 2026	Experimental design
27 November 2026	Analysis and scientific argument
11 December 2026	Forecasts, control loops and agents
22 January 2027	In-class peer review and revision
5 February 2027	Presentations and reproducibility

Fridays: work on your project

Short input together, then project work and individual support.

Bring your code, data, draft and current blocker.

Your project

One question. Teams of 3–4. A meaningful comparison.

Submit	Requirement
Paper	English, IEEE two-column, six pages including references
Artifact	Reproducible evidence
Declaration	Contributions, outside the page limit

Group choice: **12 Oct, 13:45 → 27 Nov, 23:59**.
Nominate one contact person.

What counts towards your grade?

The **paper and artifact** are assessed together. Normally one group grade.
Substantial, documented contribution differences may lead to individual grades.

Paper area	Weight
Introduction & related work	20%
Approach & methodology	25%
Evaluation & results	30%
Discussion & conclusion	15%
Abstract, writing & reproducibility	10%

Who did what?

Record individual and joint work in DECLARATION.md.

Record	Make it concrete
Paper	Sections, figures and revisions
Artifact	Code, configuration and experiments
Joint work	Shared task and each person’s contribution

Point to the work so the contribution can be checked.

What goes with the paper?

The paper makes the argument. The **artifact** lets me check it.

- Code and configurations
- Data and software versions
- Reproduction steps
- Seeds, failed runs and exclusions

Public repository: optional. Access for the examiner: required.

Scientific integrity

Follow the IEEE publishing ethics requirements.

- Cite ideas, methods, data and figures
- Check references against the original
- Report what you actually measured
- Keep counter-evidence
- State limitations and uncertainty

A result against your hypothesis is still a result.

AI use and disclosure

Follow the IEEE policy on AI-generated content.

- Spelling and grammar fixes only: disclosure optional
- Generated or substantially rewritten text: disclose
- Generated figures or code: disclose
- Name the system, affected parts and extent in the **Acknowledgments**

You are responsible for the references, code and technical claims.

Dates and deadlines

Date	Deliverable / session
27 November 2026, 23:59	Group selection → Moodle
11 January 2027, 23:59	Full draft + supporting material → Moodle
22 January 2027, in class	Read, write and hand in the paper review
5 February 2027	Presentations in class (ungraded)
12 February 2027, 23:59	Paper, artifact, declaration → Moodle

Optional peer-review bonus

22 January 2027, in class. No make-up date.

One qualifying review improves the individual grade by **one step**, e.g. 2.3 → 2.0.
Best possible grade: **1.0**. Failing grades remain unchanged.

One page, legibly handwritten, German or English.
Work independently. **No AI or other aids.**

For recognized accommodations, contact the lecturer early.

What makes a qualifying review?

A qualifying review covers four things:

- **Summary:** question and main result
- **Method:** assess a methodological choice
- **Judgment:** strength and limitation with reasons
- **Revision:** propose a feasible improvement

I record the four criteria and my reasons.

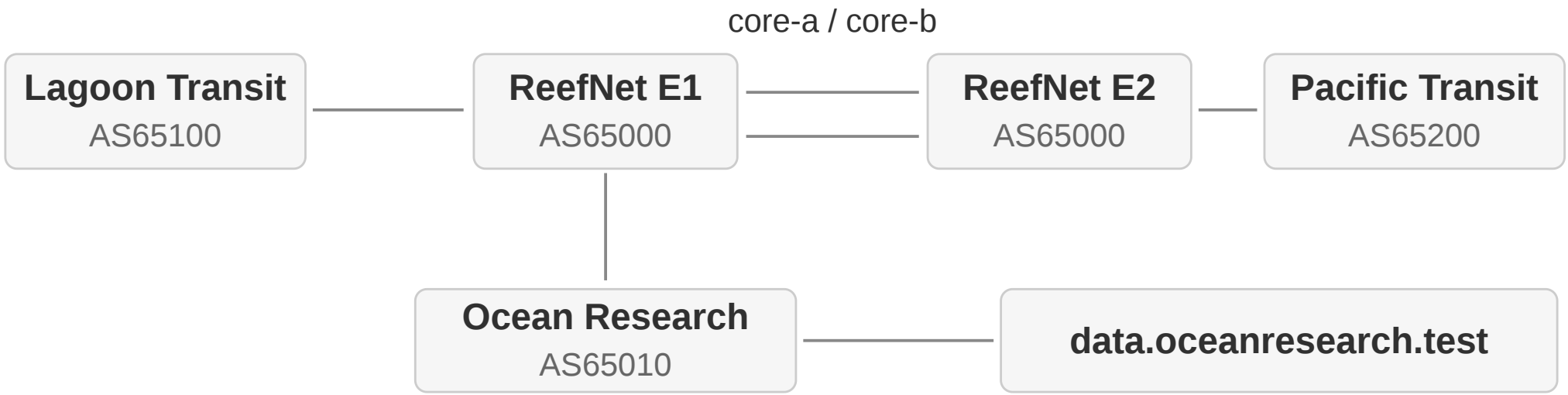
12 October 2026

Networking 101

One network carries the technical block.

Meet ReefNet at BOB1

ReefNet connects **Ocean Research** through two upstreams.



Four views. Four questions.

View	Question
NetBox	What should exist?
Router	What exists now?
Grafana	What changed over time?
External probe	Does the service work?

Follow one service request.

How does this network actually work?

Follow one service request.

In ReefNet: a Lagoon probe requests `data.oceanresearch.test`.

One request crosses several independently operated links.



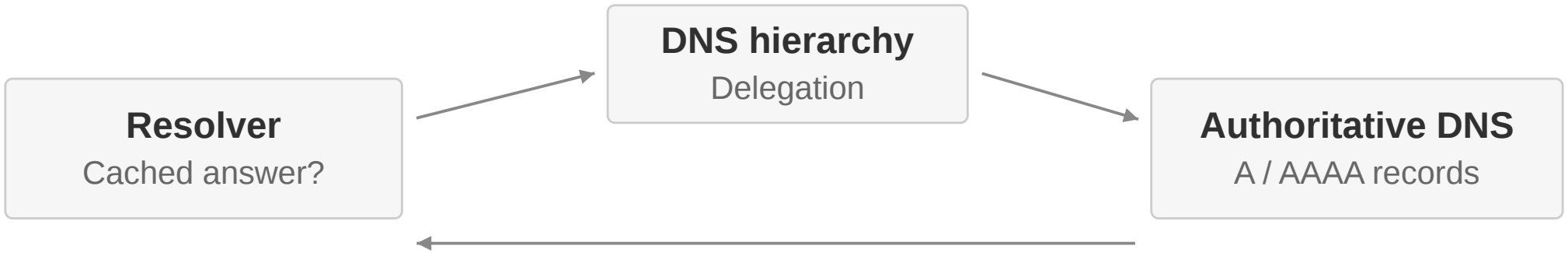
What must work before the application can return a response?

The TCP/IP stack

Application	HTTP, DNS
Transport	TCP, UDP
Internet	IP, ICMP
Link	Ethernet
Physical	Copper, fiber, radio

Each layer carries data for the layer above it.

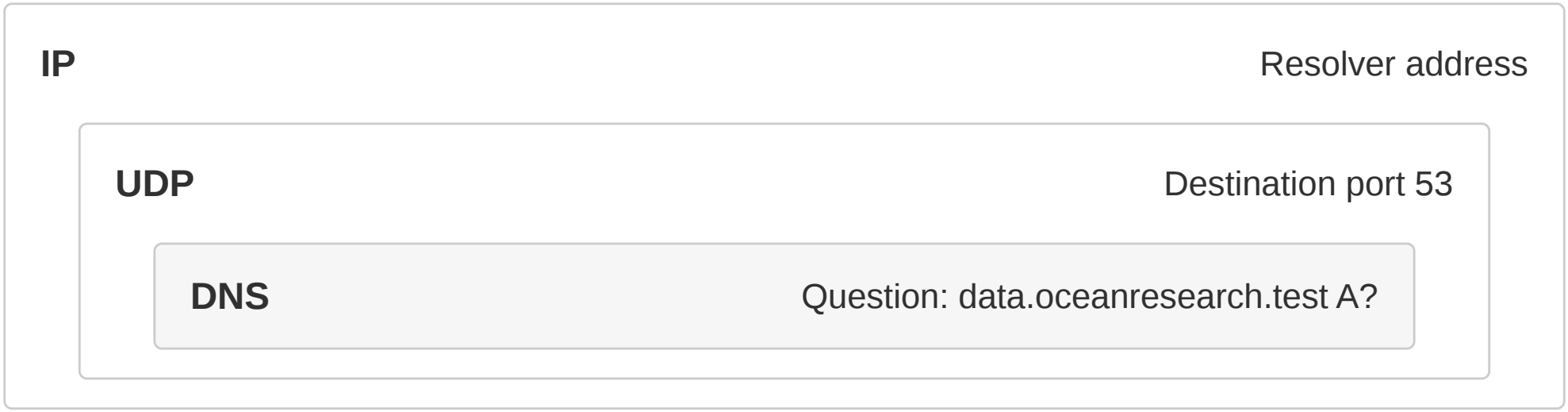
A name needs an address.



An address is an answer about naming, not service health.

A → IPv4
AAAA → IPv6.

A DNS query becomes transport data



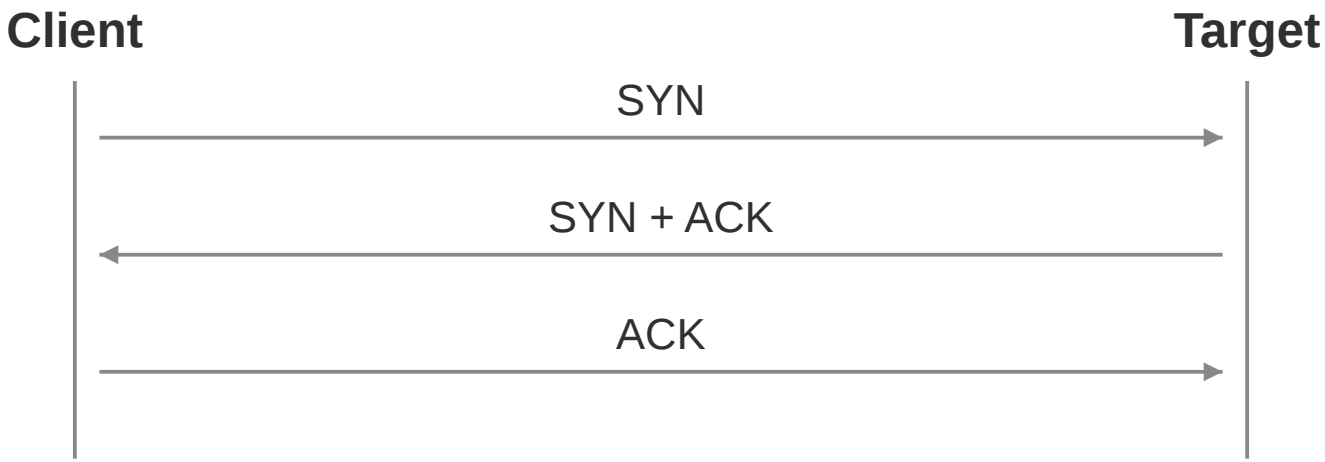
This query uses UDP. DNS also supports TCP.

TCP and UDP

	TCP	UDP
Data	Ordered byte stream	Individual datagrams
Connection	Required	None
Retransmission	Built in	Application decides

TCP and UDP add port numbers above IP.
An IP address + port identifies the transport endpoint.

TCP gets the conversation started.



TCP setup precedes the application response.

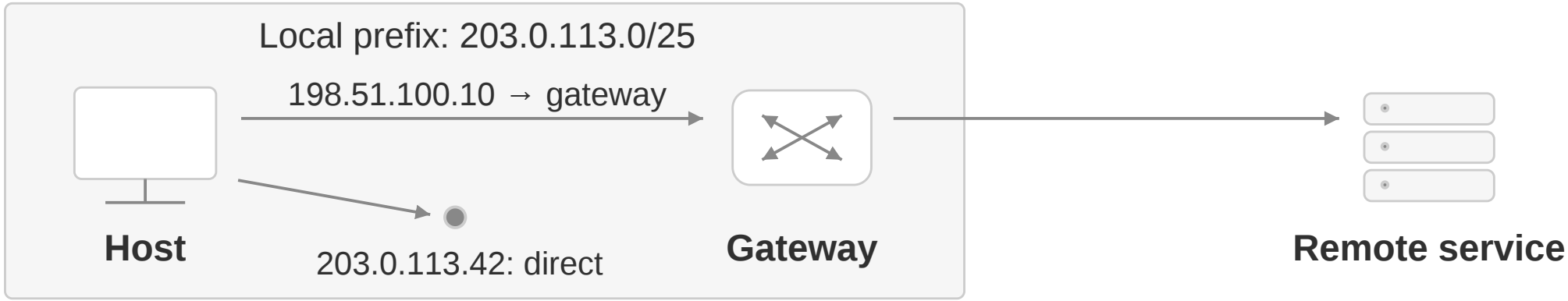
TCP orders bytes, retransmits loss and controls congestion.

IP gives the packet a destination.



IPv4 uses 32-bit addresses. IPv6 uses 128-bit addresses.
Routers use the **destination address** to look up where to forward the packet.

On-link or through a router?



The host looks up the destination in its routing table to choose an **interface**.
It sends directly to an **on-link destination**, otherwise via a **gateway**.

Use the longest matching prefix

Destination: **198.51.100.200**. Example Forwarding Information Base (FIB):

Prefix	Next hop
`0.0.0.0/0`	`192.0.2.1`
`198.51.100.0/24`	`192.0.2.2`
`198.51.100.128/25`	**`192.0.2.3`**

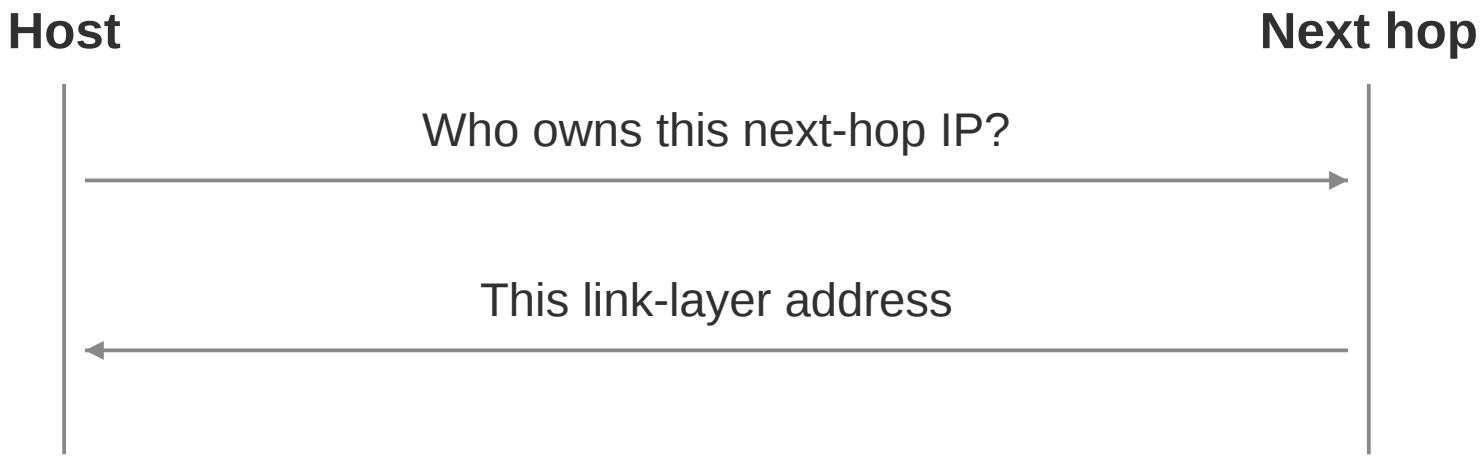
`/25` matches 25 address bits. The most specific match wins.

Ethernet carries frames across one link



An Ethernet frame uses **MAC addresses for this link**.
The IP packet still names the final destination.

Find the next-hop MAC



ARP uses broadcast. ND uses multicast solicitation.

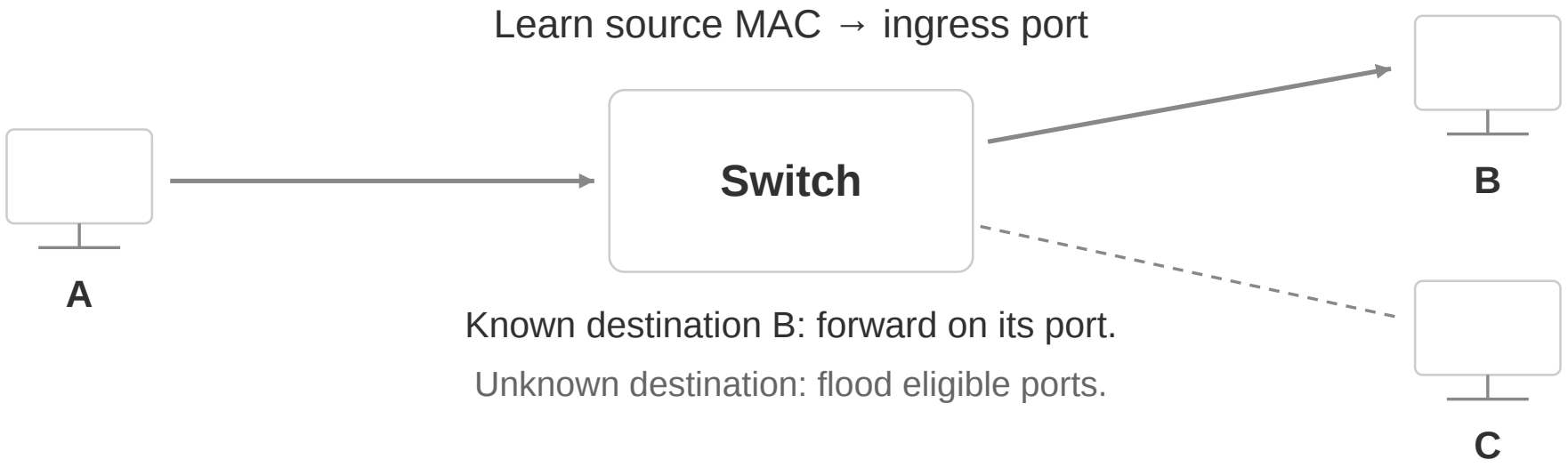
IPv4: ARP

Resolve the selected next hop’s IPv4 address to its MAC address.

IPv6: Neighbor Discovery

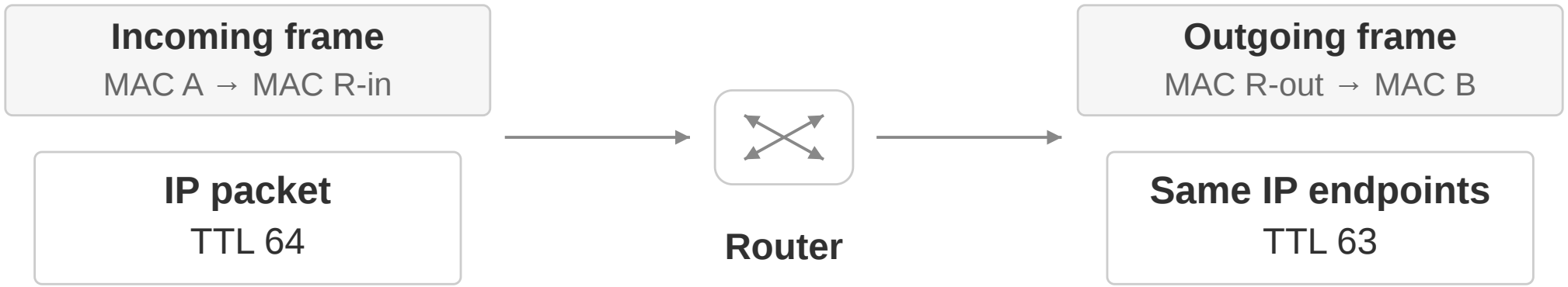
Resolve the selected next hop’s IPv6 address using ICMPv6.

Switches learn MAC addresses and forward frames



An IPv4 ARP request is broadcast. The switch floods it while learning the sender’s MAC.

Routers repeat the process hop by hop



Rebuild the frame. Keep the IP destination. TTL values are illustrative.

Each router looks up the next hop, builds a **new frame** and decreases TTL / Hop Limit. The packet keeps its **destination IP**. The link-layer MAC addresses change.

Where do routes come from?

Use the running Networking lab. Leave a Linux node with `exit`, a router with `quit`.

Route source	Origin
Connected	Addressed interface
Static	Operator configuration
Dynamic	Routing protocol

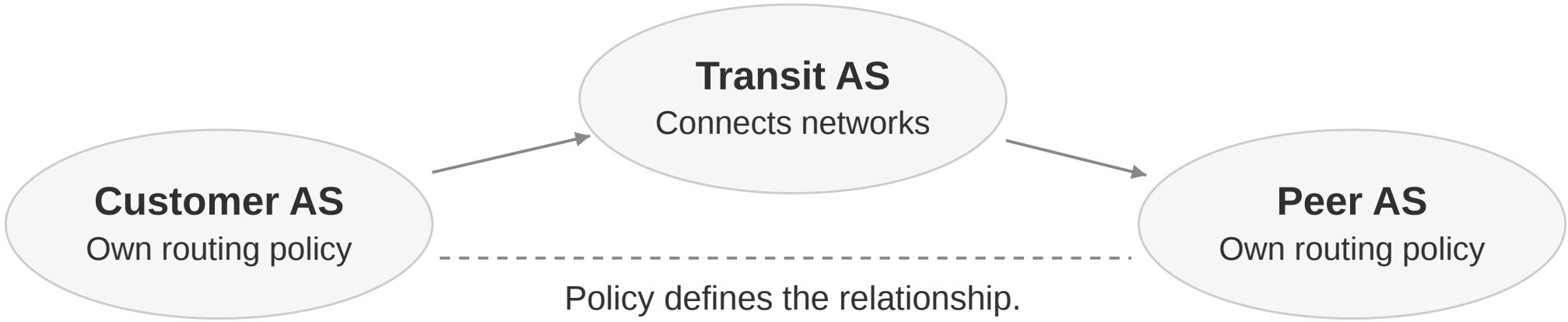
WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default route-table ipv4-unicast summary
```

An AS is an administrative routing domain

An **autonomous system (AS)** presents one routing policy externally.
An **Interior Gateway Protocol (IGP)** finds routes inside it.

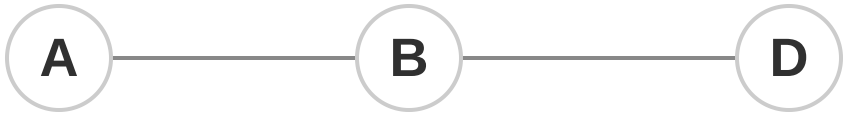


An AS is an administrative boundary, not one router.

Two ways to learn an internal route

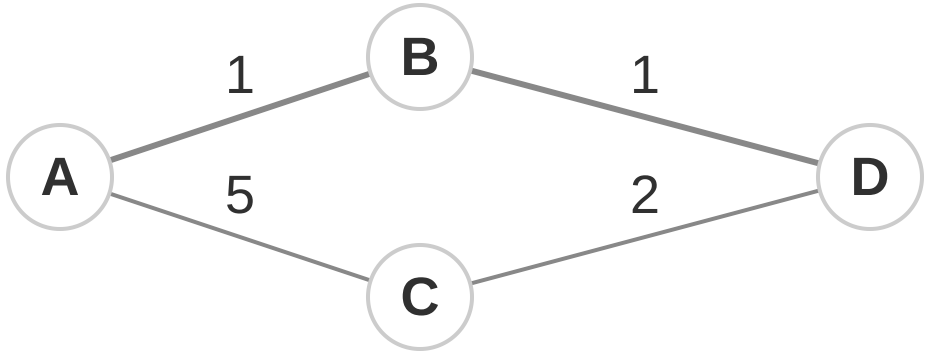
Distance vector (RIP)

B → A: “D is 1 hop away.”



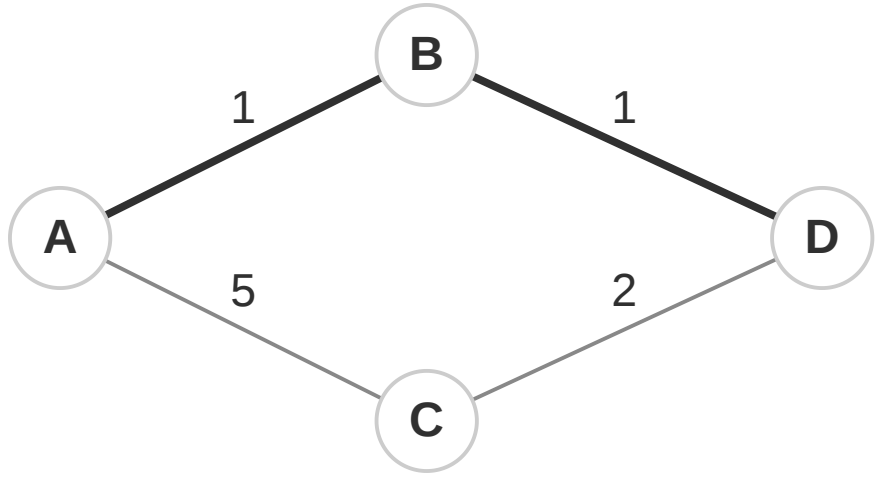
Share distances to destinations.
A adds 1 hop: D via B, 2 hops.
A advertises its own distance onward.

Link state (OSPF)



Share your own links and their costs.
Relay across the area → shared map.
Each router calculates its own paths.

OSPF: each router calculates its own paths



Example costs:
Via B: **1 + 1 = 2**. Via C: 5 + 2 = 7.

ReefNet uses OSPFv2 for IPv4 and OSPFv3 for IPv6.

Both provide paths to the iBGP loopbacks over either core link.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols ospf neighbor
```

BGP exchanges reachability.

Path vector: prefixes, AS paths and other attributes.

eBGP

Routes between ASes.

iBGP

Routes within an AS.
The IGP is still needed.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp neighbor
show network-instance default protocols bgp routes ipv4 summary
```

BGP separates route families

AFI (Address Family Identifier): IPv4 or IPv6.

SAFI (Subsequent Address Family Identifier): e.g. unicast.

In ReefNet	Routes to destinations such as
`ipv4-unicast`	`198.51.100.0/24`
`ipv6-unicast`	`2001:db8:100::/48`

Established confirms the session is up. Check which families exchange routes.

BGP is policy first.

For comparable routes:

1. Apply local policy.
2. Prefer higher local preference.
3. Compare AS_PATH and further tie-breakers.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
```

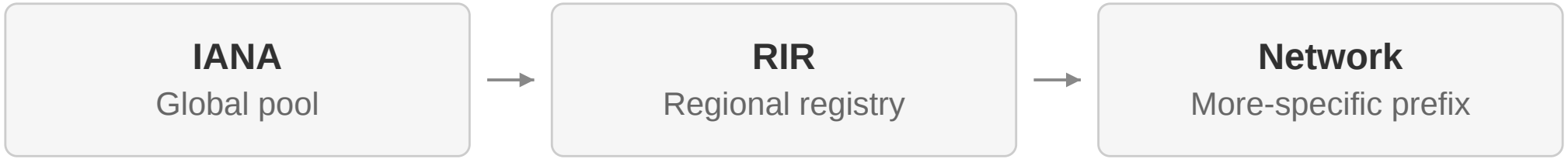
A longer path can be selected on purpose.

Transit and peering

	Transit	Peering
Reachability	Provider supplies broader reachability	Networks exchange selected routes
Commercial relation	Customer pays provider	Often settlement-free

An **IXP** provides shared infrastructure for peering.
Export policy determines the actual routes exchanged.

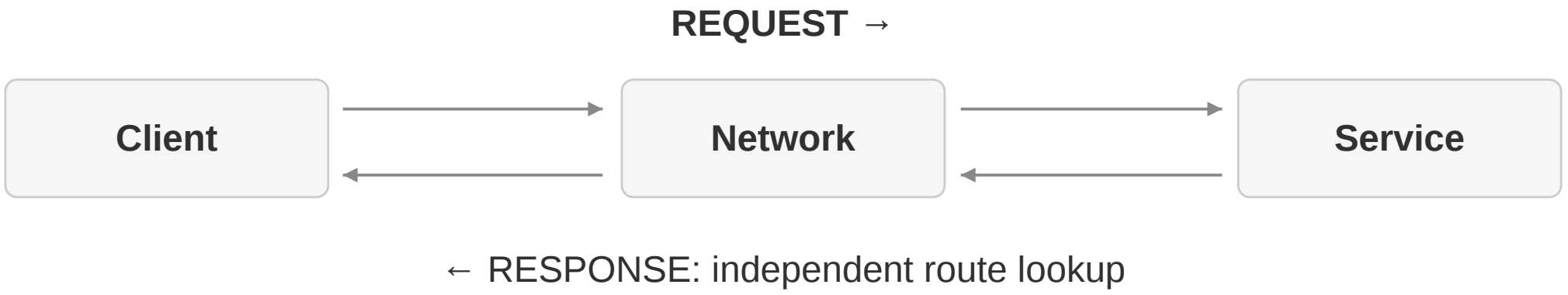
Who assigns the prefixes we route?



Administrative delegation, not a forwarding path

BOB1 uses documentation prefixes. Its service is not on the public Internet.

The response needs its own route.



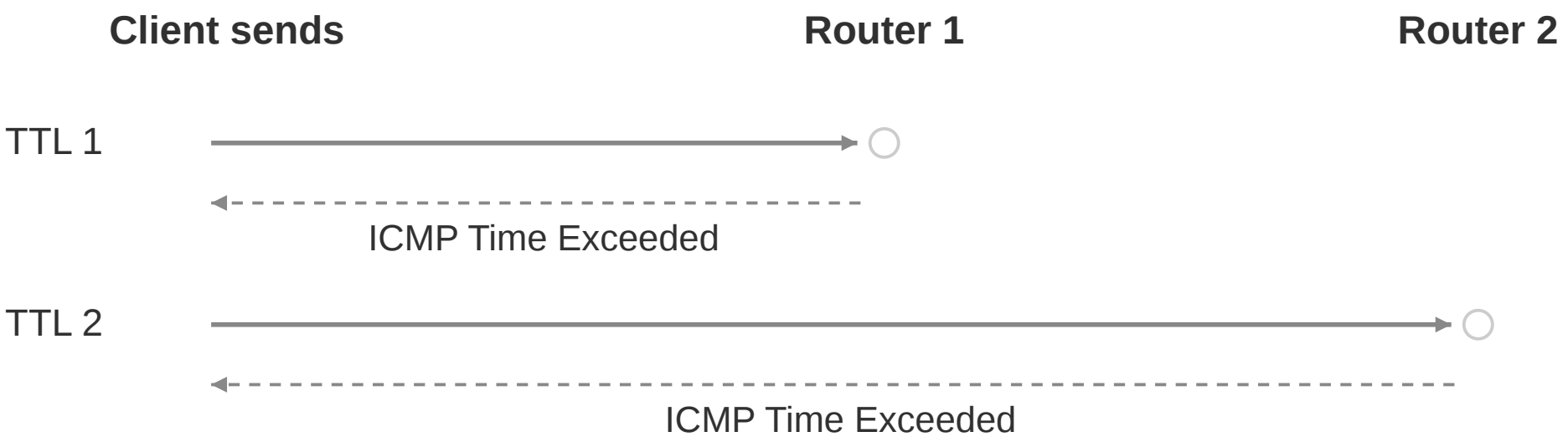
Reachability needs both directions. Paths may be asymmetric.

ICMP: errors and diagnostics

Message	Meaning
Echo Request / Reply	Ping
Destination Unreachable	Cannot deliver
Time Exceeded	TTL / Hop Limit expired
Packet Too Big (IPv6)	Exceeds link MTU

A ping reply proves IP reachability, not application health.

Traceroute expires TTL / Hop Limit.



Larger TTL (IPv4) or Hop Limit (IPv6) values reveal responding routers.
A missing reply does not identify the failed forwarding hop.

Even a correct path needs healthy links

Medium	Signal	Fault evidence
Copper	Electrical	Link loss, FCS errors
Fiber	Light	Low receive power, FCS errors

A path may be correctly routed and still fail on a degraded link.
Rising **FCS errors** can be evidence even while the link stays up.

Follow the request

How does a request from Lagoon reach `data.oceanresearch.test`,
and how does the reply get back?

60 minutes: 30 to clone the repo and run `make setup`, then 30 to explore.

After `READY FOR CLASS`, in WSL on the lab PC:

```
make networking
make task
```

Check addresses, DNS and HTTP. Sketch the path.
Then try Pacific and IPv6.

Exercise overview

What did your checks tell you?

- Which address came from DNS?
- Which address was your next hop?
- How did you check the return path?
- What did HTTP tell you that ping could not?

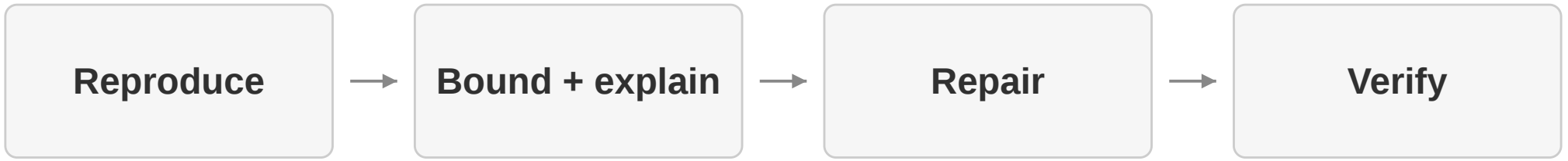
Bring one result you can explain and one question that is still open.

Network Operations

The service is broken.

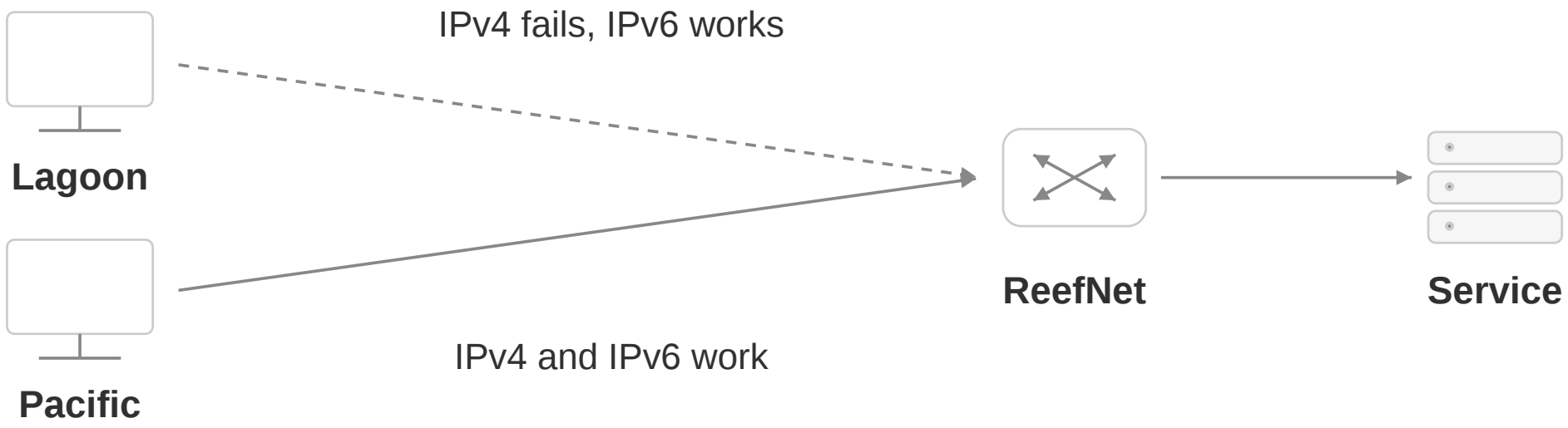
What happened, and how do we restore it safely?

One incident, from report to recovery.



Evidence before action. Recovery before closure.

A customer reports partial reachability



Record the source, address family, protocol and time.

Who cannot reach the service?

WSL `make enter NODE=probe01.bob1.lagoontransit.test`

Inside the Lagoon probe

```
ping -4 -c 2 -W 1 198.51.100.10
ping -6 -c 2 -W 1 2001:db8:100::10
```

Use `exit`, then in WSL:

```
make enter NODE=probe01.bob1.pacifictransit.test
```

Repeat both tests. Which path and address family fail?

Bound the incident.

	IPv4	IPv6
Lagoon	**FAIL**	**PASS**
Pacific	**PASS**	**PASS**

Keep the service and test method fixed. Vary the viewpoint.

Separate state from service.

Management Plane	Configure and monitor	CLI, gNMI, SNMP
Control Plane	Learn and select routes	OSPF, BGP
Data Plane	Forward packets	IP packets, Ethernet frames

Management access and established routing sessions do not prove service delivery. Verify the service from the customer's viewpoint.

Is BGP actually established?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp neighbor
```

IPv4 neighbor	Role
`192.0.2.0`	Ocean Research
`192.0.2.2`	Lagoon Transit
`10.255.0.2`	ReefNet edge02 iBGP

Did the customer route arrive?

Compare the customer prefix in both address families.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
show network-instance default protocols bgp routes ipv6 prefix 2001:db8:100::/48
```

A route in BGP is not yet proof of forwarding.

Is IPv4 installed for forwarding?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default route-table ipv4-unicast prefix 198.51.100.0/24
```

Active route → resolved next hop → outgoing interface

Compare the advertisements

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI: toward Lagoon

```
show network-instance default protocols bgp neighbor 192.0.2.2 advertised-routes ipv4
```

Use `quit` before entering the other router.

WSL `make enter NODE=edge02.bob1.reefnet.test`

Router CLI: toward Pacific

```
show network-instance default protocols bgp neighbor 192.0.2.5 advertised-routes ipv4
```

Find `198.51.100.0/24`.

You are on call

A user behind Lagoon cannot reach the Ocean Research service.

In WSL:

```
make operations
make task
```

45 minutes: find the fault, repair it and verify recovery.
Keep your commands and evidence. We compare diagnoses before repairing.

Exercise overview

Which policy does this neighbor use?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
info from running network-instance default protocols bgp group lagoon-v4
info from running network-instance default protocols bgp group lagoon-v6
```

What is attached only to IPv4?

Remove the override, then check the service

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
enter candidate
delete / network-instance default protocols bgp group lagoon-v4 export-policy
diff
commit now
```

The group inherits `EXPORT-BGP`. Leave the other paths unchanged.

The change is not the recovery proof.

Leave the router with `quit`.

WSL `make enter NODE=probe01.bob1.lagoontransit.test`

Inside the Lagoon probe

```
curl --fail --max-time 5 -4 http://198.51.100.10/
curl --fail --max-time 5 -g -6 'http://[2001:db8:100::10]/'
```

Use `exit`, then in WSL:

```
make enter NODE=probe01.bob1.pacifictransit.test
```

Repeat both HTTP checks. Return with `exit`, then run `make test`.

Record impact, evidence and recovery

A useful update states:

impact → evidence → action → verification

For example:

> IPv4 through Lagoon Transit was missing because the customer route was filtered on export. The policy attachment was removed and reachability now succeeds from both external probes. IPv6 was unaffected.

Write an RFO after the incident

RFO = Reason for Outage

Record	ReefNet incident
Impact	Lagoon IPv4 unreachable
Cause	Wrong export policy on `lagoon-v4`
Recovery	Override removed, HTTP verified
Prevention	Test IPv4 and IPv6 exports

Record unknown times as **unknown**.

FCAPS: five management areas

- **Fault**: detect, isolate and recover
- **Configuration**: control intended state
- **Accounting**: attribute resource use
- **Performance**: measure service and capacity
- **Security**: control access and change

Our repair touched several areas at once.

Stop here, repeat later

Keep notes on your checks, findings and next step.

`make down` removes the running lab. **Router changes are discarded.**

`make operations` starts the **original incident**, not your last state.

On your own Windows PC, install WSL-Containerlab, then follow the lab setup.
Use your notes to repeat the investigation.

13 October 2026

Automation + Incidents

Yesterday we examined a manual network repair.

Today we need to make the same class of change **safely and repeatedly**.

At the end of today: **Who Broke the Internet?**

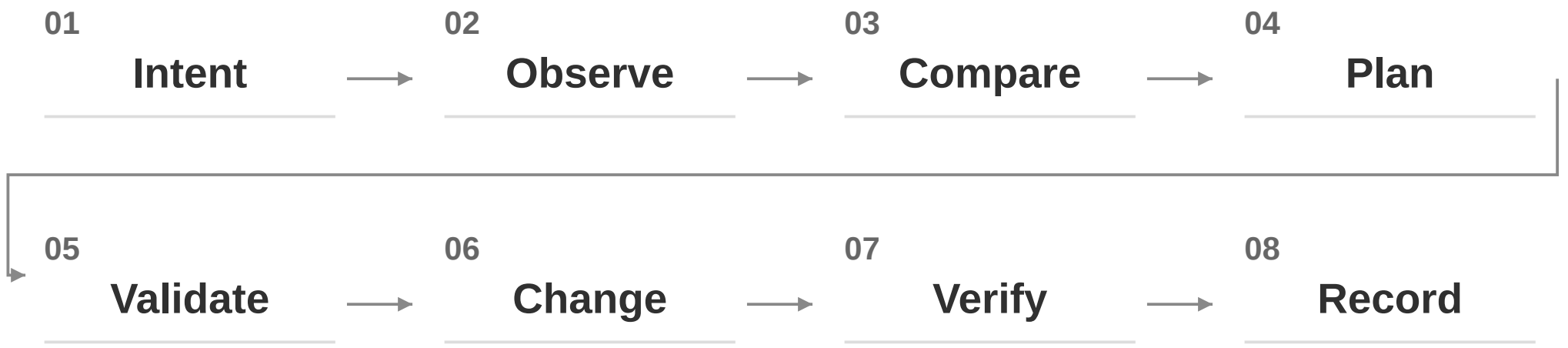
Same incident. This time, use an API.

Yesterday we repaired a router through its CLI.
Today we will use NetBox and gNMI.

How do we choose the right device, make the change and check it worked?

The exercise starts with `make automation`.
You can begin even if you did not finish Operations.

The safe change loop



Failed verification: record actual state, back out, verify again.

YANG, NETCONF and gNMI support this workflow.

Agree on a change contract.

Boundary	Agree before execution
Scope	Target and permitted diff
Entry	Preconditions and reviewed input
Exit	Service checks and deadline
Recovery	Backout, owner and access path

The ReefNet change contract

Target: **ReefNet Edge 01**, BGP group `lagoon-v4`.

	Required state
Before	Override: `BLOCK-CUSTOMER-V4`
After	Inherit `EXPORT-BGP` without a group override
Preserve	Lagoon IPv6 and Pacific Transit
Prove	Customer IPv4 export and external reachability

Which facts need a fresh read from the router?

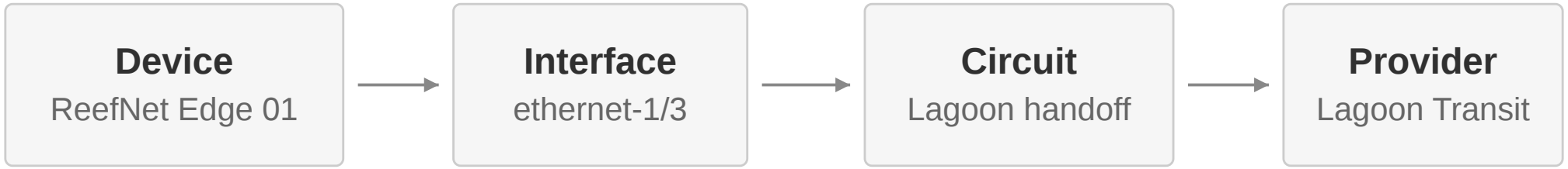
Intent, policy, configuration

Level	ReefNet example
Intent	Customer reachable through both upstreams
Policy	Lagoon IPv4 inherits `EXPORT-BGP`
Configuration	`lagoon-v4` currently has a group override

A correct policy on the wrong group still fails the intent.

Inventory gives the target context.

An inventory is a graph of relationships.



Join the intended circuit to the actual change target.

Read the device inventory.

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

```
TOKEN=$(cat /state/netbox-token)
NETBOX_API=http://netbox.bob1.reefnet.test:8080/api
curl -fsSG -H "Authorization: Bearer $TOKEN" \
  "$NETBOX_API/dcim/devices/" \
  --data-urlencode name=edge01.bob1.reefnet.test |
  jq '{count, devices: [.results[] | {name, primary_ip4}]}'
```

Expect one device. Keep the token private.

Read the circuit intent too

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

```
TOKEN=$(cat /state/netbox-token)
NETBOX_API=http://netbox.bob1.reefnet.test:8080/api
curl -fsSG -H "Authorization: Bearer $TOKEN" \
  "$NETBOX_API/circuits/circuits/" \
  --data-urlencode cid=BOB1-LAGOON-001 |
  jq '.results[] | {cid, provider: .provider.name, status: .status.label}'
```

Compare circuit intent with live router state.

Join inventory into one change input

Example API reads:

```
GET /api/dcim/devices/?name=edge01.bob1.reefnet.test
GET /api/dcim/interfaces/?device=edge01.bob1.reefnet.test
GET /api/ipam/prefixes/?prefix=198.51.100.0%2F24
GET /api/ipam/asns/?asn=65100
```

Join these objects into one approved change input.

Intent and observation can drift.

Compare the same **scope**, **time** and **meaning**.

Stale data or different representations can look like drift.
Check those before reconciling.

Unavailable input is not an empty list.

Input	State
Inventory snapshot	Current
Monitoring API	Unavailable
Previous output	Known working

Stop publication or use an approved, age-limited fallback.
Unknown targets must not silently disappear.

Freeze the input you reviewed

```
{
  "change_id": "CHG-017",
  "device": "edge01.bob1.reefnet.test",
  "bgp_group": "lagoon-v4",
  "expected_override": "BLOCK-CUSTOMER-V4",
  "action": "delete group export-policy",
  "expected_inherited_policy": "EXPORT-BGP"
}
```

Reviewer and executor need the **same approved input**.

Also keep the current state needed for backout.

Choose the automation mechanism.

Layer	Examples
CLI automation	Netmiko
Data models	YANG, OpenConfig
Model-driven protocols	NETCONF, gNMI

The same preconditions, service checks and recovery contract still apply.

Netmiko: automate the CLI you already know

Illustrative SSH read on SR Linux:

```
from netmiko import ConnectHandler

with ConnectHandler(**device) as conn:
    output = conn.send_command("show interface")
```

Netmiko handles the session. You interpret the result.

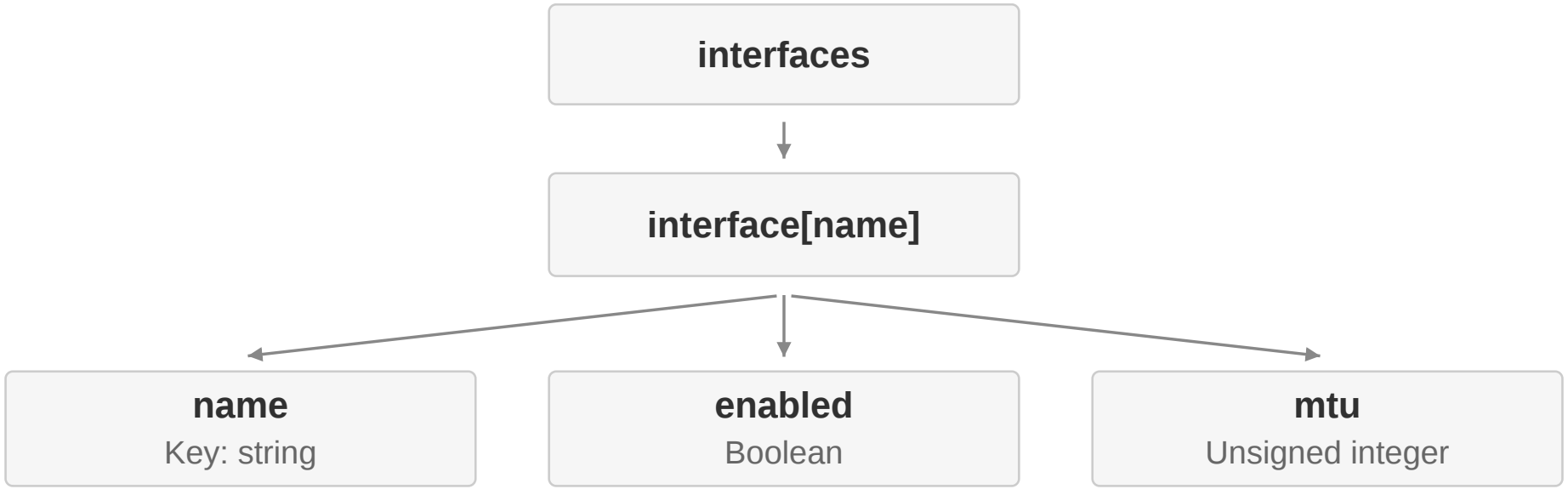
A prompt is not a postcheck.

Illustrative CLI error:

```
router(config)# interface does-not-exist
% Invalid interface
router#
```

The session survived. The command did not.
Read back state and check the service.

YANG describes management data.



The model constrains data. The change contract constrains behavior.

YANG keys and constraints

```
list interface {
  key "name";
  leaf name { type string; }
  leaf enabled { type boolean; default true; }
  leaf mtu { type uint16 { range "1280..9216"; } }
}
```

A module contains this fragment. The MTU range is illustrative.

A list key must name an existing child leaf.

Enabled, but no carrier

View	Interface fact
Configuration	Interface is enabled
Observed state	Carrier is down

Configuration and observed state answer different questions.

Check the physical link and the peer before changing the configuration.

OpenConfig

Vendor-neutral YANG models for interfaces, routing and telemetry.

A common model does not guarantee identical device support.
Check supported paths and operations.

NETCONF edits a datastore.

With a supported candidate datastore:

1. Lock, edit and validate.
2. Commit.
3. Verify the service independently.
4. Unlock and record.

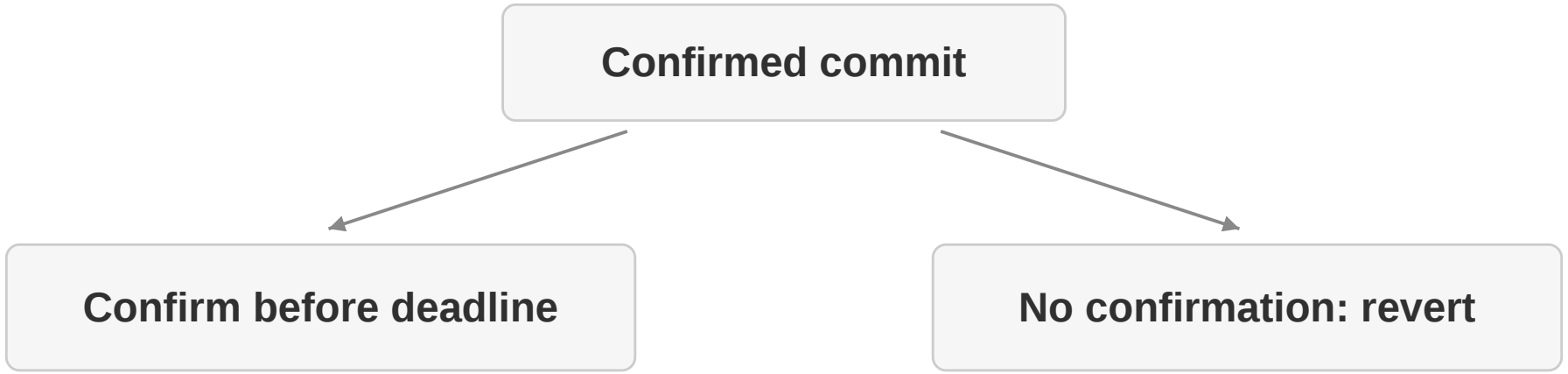
Check the advertised capabilities.

Read the NETCONF error.

```
<rpc-error>
  <error-type>application</error-type>
  <error-tag>invalid-value</error-tag>
  <error-severity>error</error-severity>
</rpc-error>
```

Inspect the tag, path, message and datastore state after failure.
Then decide whether a retry is safe.

Confirmed commit

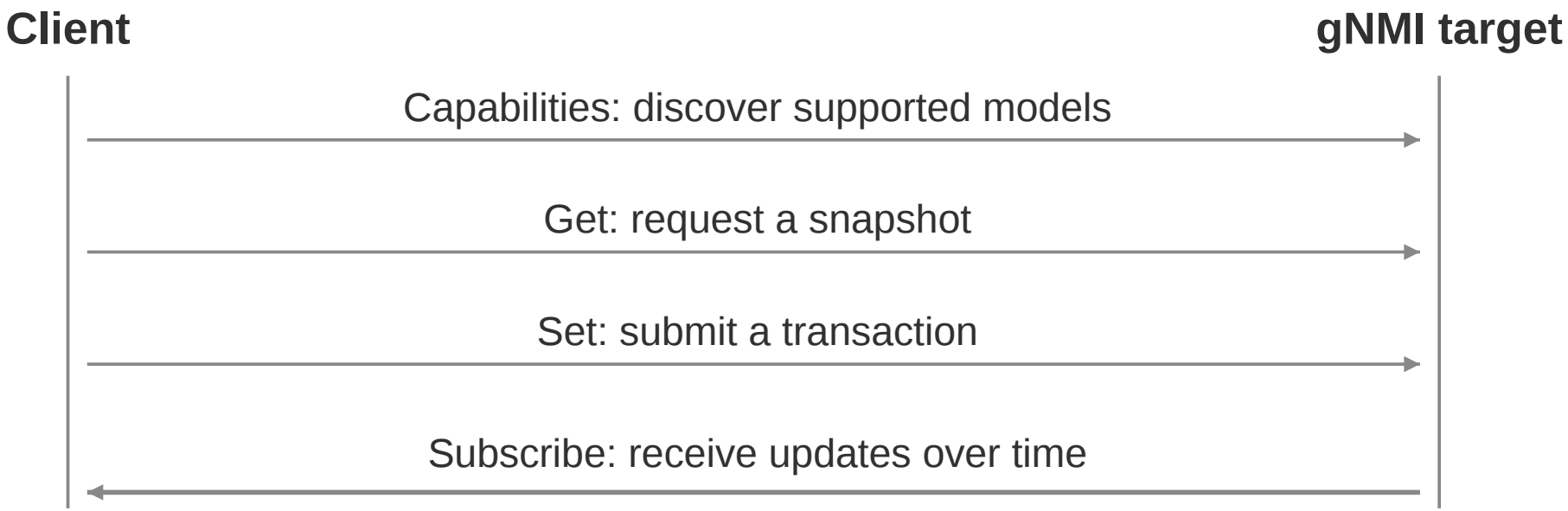


Verify service before confirming. Requires target support.

Who confirms if automation loses contact? Test the revert path.

gNMI defines four RPCs.

RPC = remote procedure call



Read a gNMI path

```
/interface[name=ethernet-1/1]/statistics
```

- `name=ethernet-1/1` selects one interface.
- `statistics` selects its subtree.
- The **target** identifies the router.

gNMI Set

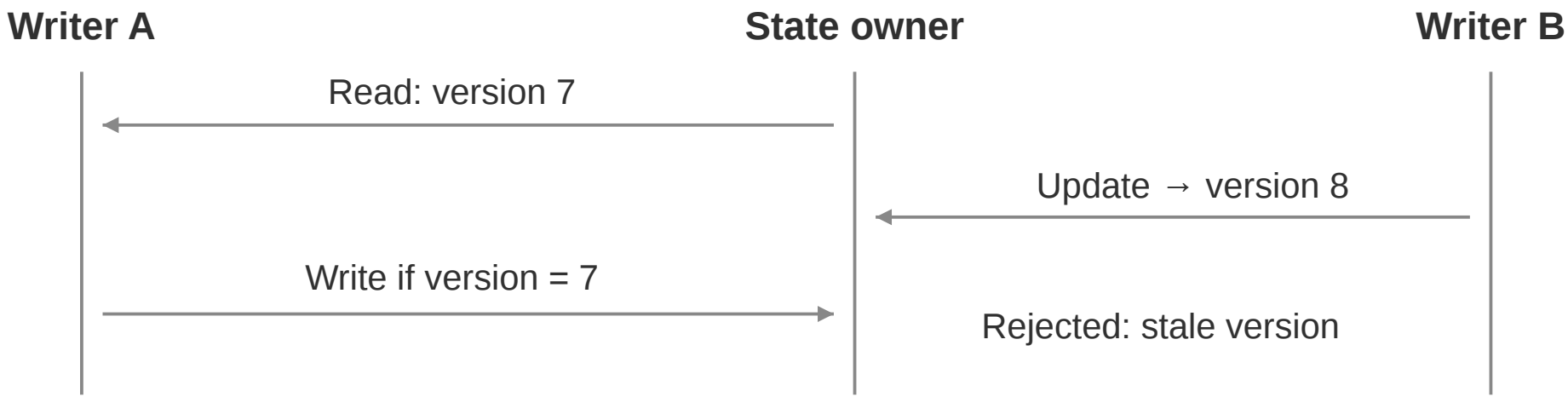
One request, one target, one transaction.

Result	Target behavior
Success	Apply all requested changes
Failure	Restore the pre-request state
Timeout	Outcome unknown to the client

Read back after a timeout. Verify the service after a change.

Prevent a lost update

Optimistic concurrency: write only if the version you reviewed is still current.



Check and write must be atomic at the state owner.

A CLI precheck is useful, but it cannot make the later write atomic.

Validate before rendering

Check	Prevents
Device + BGP group	Wrong target
Current override	Stale assumptions
Import or export	Wrong direction
Inherited policy	Unsafe fallback

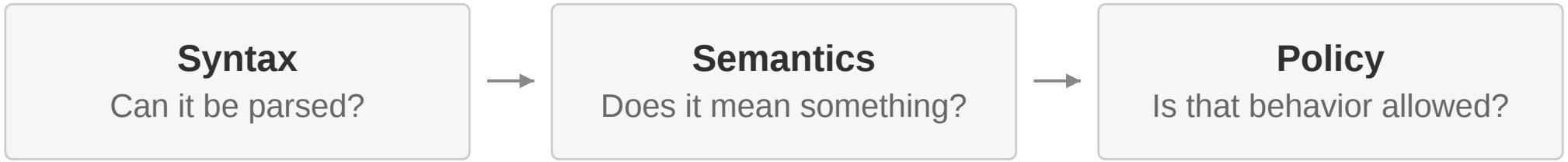
Generate and inspect the candidate.

Reviewed change **CHG-017**:

```
- export-policy [ BLOCK-CUSTOMER-V4 ]
```

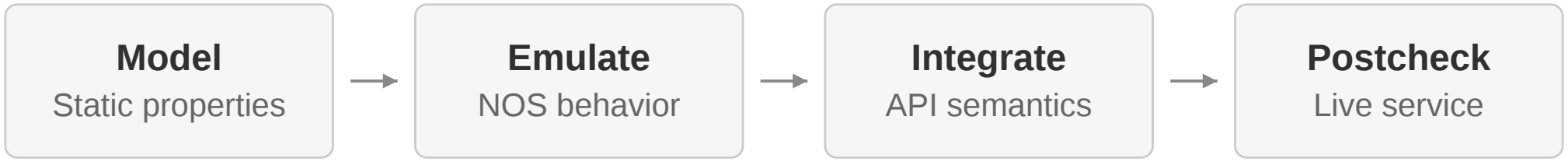
The group inherits `EXPORT-BGP`. Reject unrelated changes.

Validation has three layers.



A valid configuration can still violate the change contract.

Model it, emulate it, then check the service

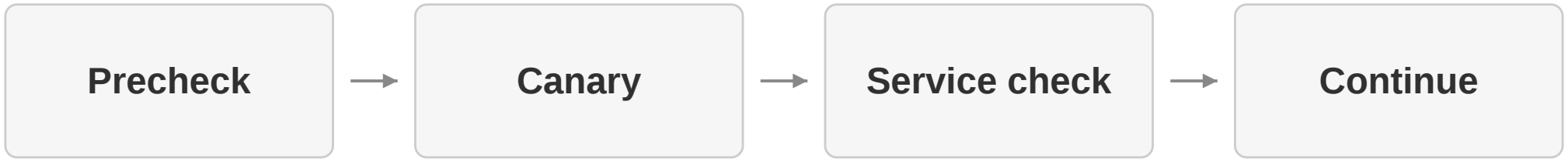


Different tests expose different failure classes.

Passing a static model check cannot prove target API behavior.

Plan the maintenance window

Method of Procedure (MOP): steps, checks and recovery ownership.



Stop time + recovery owner apply at every gate.

Plan the repair verification

Check	Expected
Lagoon: `198.51.100.0/24`	Export restored
Lagoon: `2001:db8:100::/48`	Still exported
Pacific IPv4 path	Still healthy
Both probes, both families	Service responds

Read back the route export. Test service delivery from both external probes.

Review a change plan

Eight minutes in groups of 3–4.

Aspect	Proposed plan
Input	Yesterday’s inventory
Validation	Syntax only
Rollout	All routers at once
Success	Exit code 0
Recovery	Rerun the change

Agree on **two improvements** and their acceptance checks.

Repair it through gNMI

Find ReefNet Edge 01 in NetBox. Read its configuration and justify a change.

In WSL:

```
make automation
make task
```

60 minutes: repair through gNMI and verify the service from both probes.
Use `make hint` for investigation commands.

Exercise overview

Did the API repair the service?

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01, after verifying the policy

```
BGP_PATH='/network-instance[name=default]/protocols/bgp'
POLICY_PATH="$BGP_PATH/group[group-name=lagoon-v4]/export-policy"
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSrl1!' --skip-verify --encoding json_ietf \
  set --delete "$POLICY_PATH"
```

Read back the group. Check Lagoon’s route and HTTP from both probes.

Job success is not service success.

Exit code: 0

Probe: timeout

Read back state and test the service before declaring recovery.

Beyond one router: execute and recover

What happens when a change spans multiple routers or a write has an unknown outcome?

Bound the rollout.

1. Re-read live preconditions.
2. Change one representative canary.
3. Verify the service before expanding.

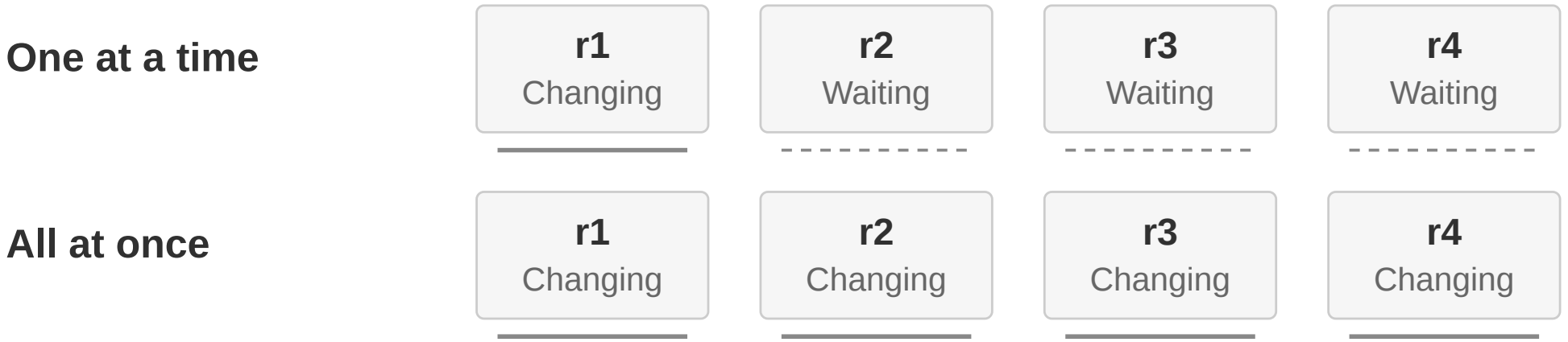
Set a stop time. Keep the backout path reachable.

Give automation a bounded identity.

Identity	Read	Write
Collector	Required state	No
Change service	Prechecks + verification	Approved scope
Reviewer	Evidence	No

Load credentials at runtime. Record the actor, never secrets.

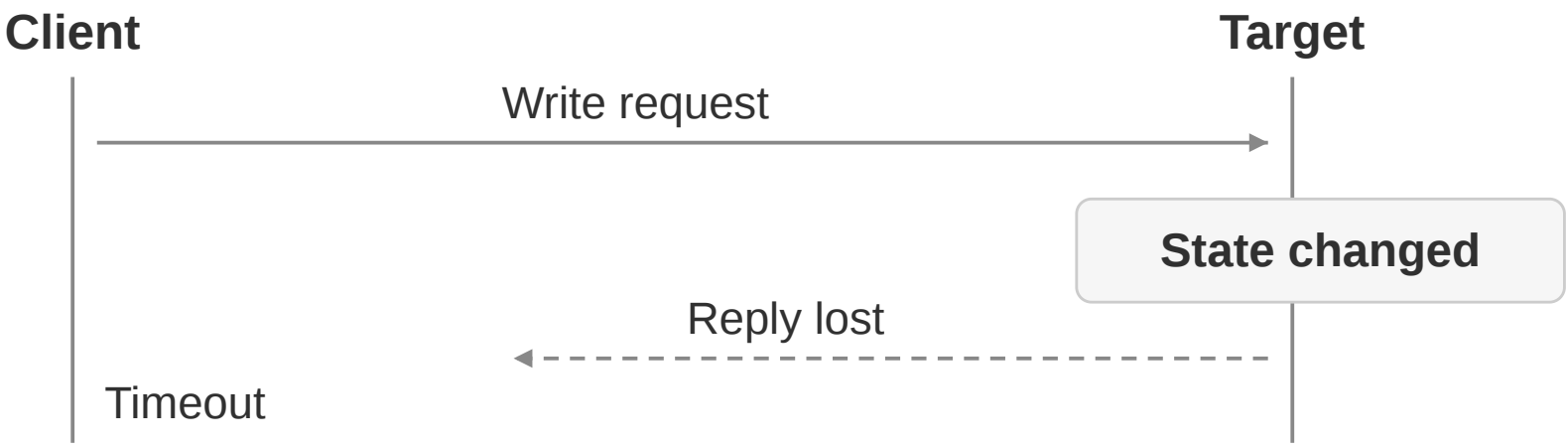
A fleet changes the failure model.



How many devices can this change affect before the next check?

More concurrency increases throughput and blast radius.

The write may already have happened



Unknown outcome ≠ no change. Read before retrying.

Record an unknown outcome explicitly in the audit trail.

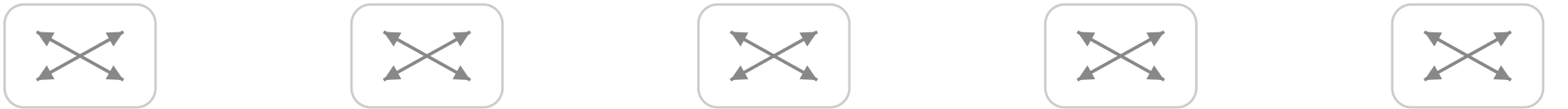
Run it twice. What changes?

Idempotence, for a fixed operation F and state s:

$$F(F(s)) = F(s)$$

Repeating the operation leaves the same state.
A retry still needs a fresh read and service verification.

One router timed out after the write

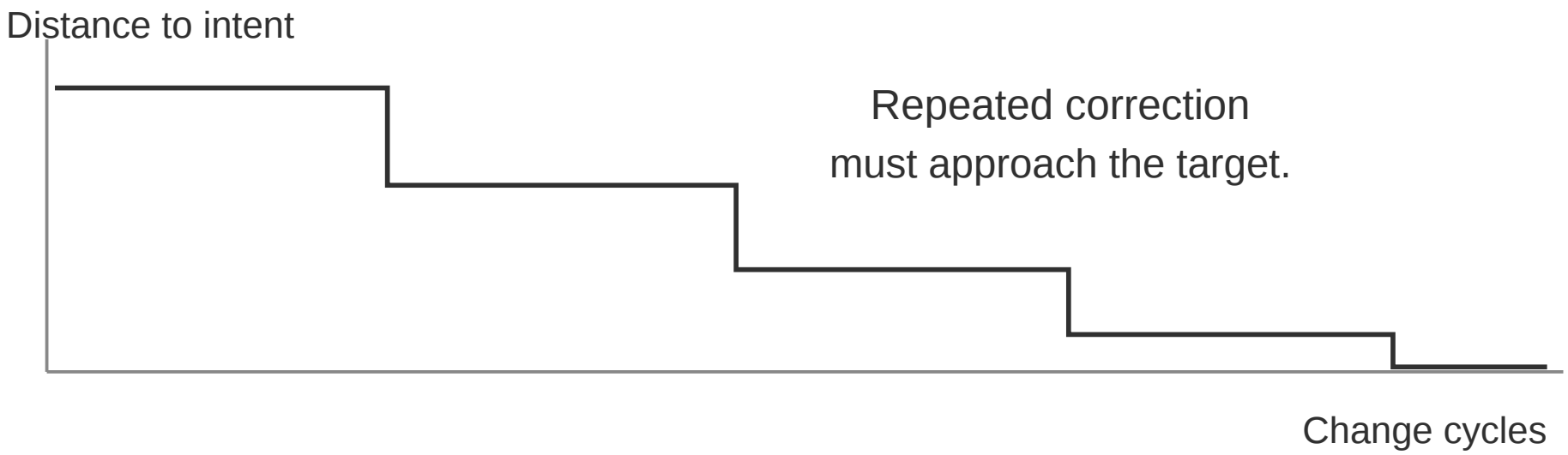


r1	r2	r3	r4	r5
Verified	Verified	Timed out	Waiting	Waiting

Progress is per device. A timeout is not a rollback.

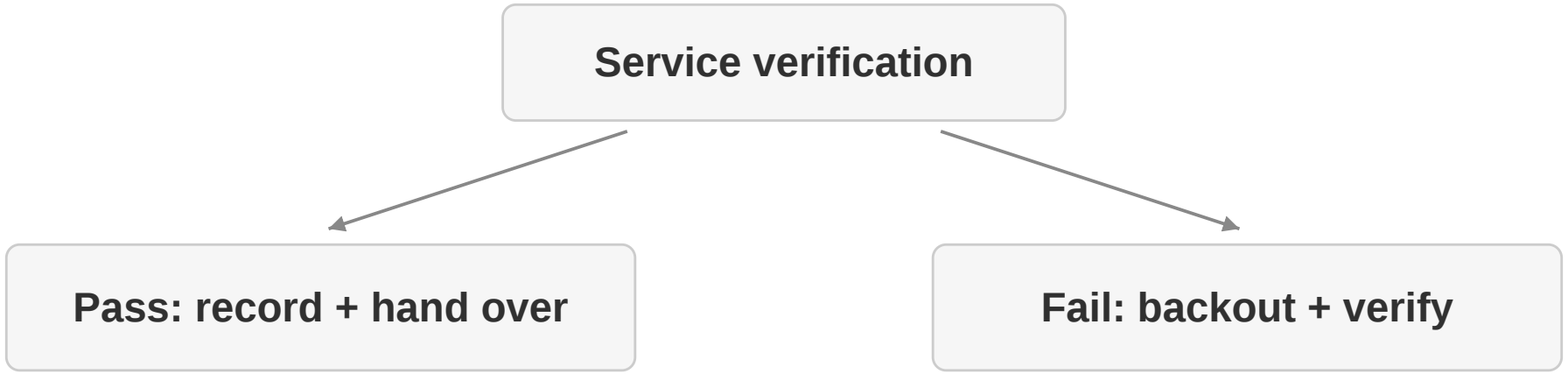
Which device needs a fresh read before any retry?

Does the loop converge?



Idempotence alone does not prove convergence.

Rollback and backout



Rollback: configuration. Backout: recovery procedure.

Record the backup's identity, time and version.
Assign an owner to the full recovery procedure.

Which state are we restoring?

“Restore last Tuesday” is not a precise instruction.

Golden configuration	Backup
Approved baseline	Captured device state

Keep identity, time, version and checksum outside the device.

Keep an audit trail.

```
{
  "actor": "course-automation",
  "target": "edge01.bob1.reefnet.test",
  "change_id": "CHG-017",
  "deleted_override": "BLOCK-CUSTOMER-V4",
  "inherited_policy": "EXPORT-BGP",
  "lagoon_ipv4_probe_passed": true
}
```

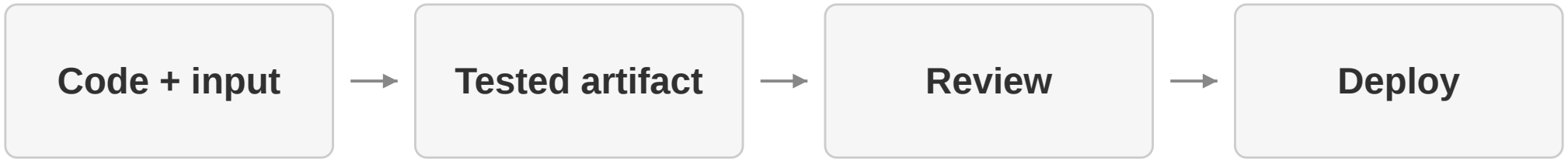
Keep timestamps and before/after evidence. Exclude credentials.

Who operates the automation?

- A named owner and bounded runtime
- One writer per target with explicit retry rules
- Fresh inputs and verified outcomes
- Tested recovery and a retained audit trail

The next shift must be able to run it.

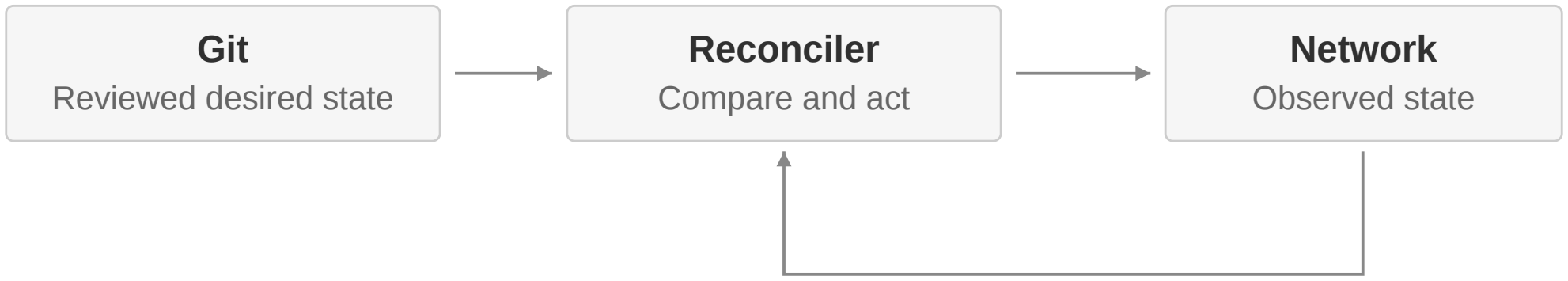
Promote the tested artifact



Deploy the reviewed artifact without regenerating it.

Approval applies to a specific code revision and input snapshot.

GitOps gives us a reviewed desired state



Fresh evidence closes the loop.

Automation: close the loop.

A write times out. **What state is the network in?**

Two minutes:

- Name the next read.
- Name the independent service check.
- Name the evidence for the next operator.

Note the original policy, your change and the verification results.
Restart the exercise later with `make automation`.

Who Broke the Internet?

Everything is redundant until the shared dependency breaks.

Which safeguard would have changed the outcome?

The rules of the show

For each case:

1. who lost what, and for how long
2. the failure boundary
3. evidence that distinguishes the cause
4. the recovery dependency
5. one safeguard worth testing

We are studying mechanisms, not awarding blame.

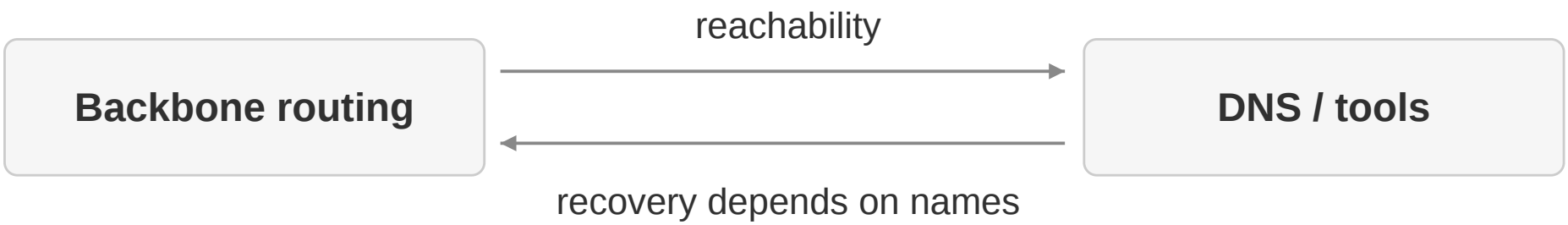
Facebook: the backbone took DNS with it

4 October 2021

1. An audit-tool bug allowed a maintenance command to disconnect the backbone.
2. DNS sites lost data-center reachability and withdrew BGP routes.
3. Running DNS servers became unreachable.

Facebook, Instagram and WhatsApp went offline.

Facebook: recovery lost its own tools



Break the loop before the outage.

Normal and out-of-band access failed. Engineers had to go onsite.

They restored the backbone, then brought services back under controlled load.

Can you still reach the router?

Management path	Uses
In-band	Production network
Out-of-band	Separate device or console access

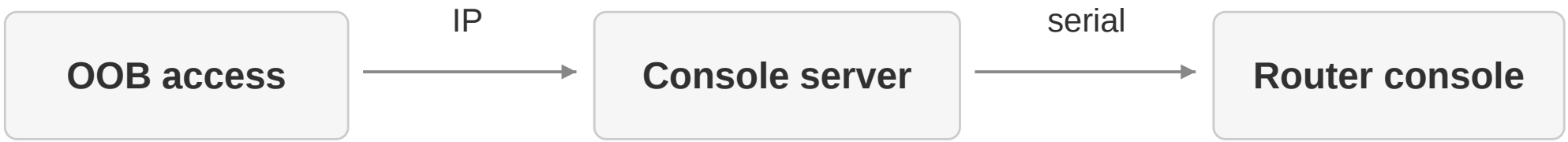
Check shared dependencies: **power, fiber, DNS and authentication**.

A console port is not a management port

Port	Needs
Ethernet management	IP connectivity and management service
Serial console	Cable, serial settings and responsive console

A **console server** provides remote serial access.
Serial bypasses IP forwarding, but still needs power.

Production recovery example: a console path



Power, emergency authentication and reachable credentials still matter.

This production recovery path is not emulated by BOB1.

Test login and repair while the production path is unavailable.

Task: the console is reachable, login fails

VPN / OOB	Connected
Console server	Logged in
Router console	Prompt visible
Remote AAA	Timeout

Exercise: AAA uses the failed backbone. Which fallback must exist?

Recovery only works if it was prepared

Dependency	Prepare
Transit	Circuit and provider contact
Console	Server, port and serial settings
Login	Tested emergency credentials
DNS / secrets	Reachable during the outage
Power	Backup along the recovery path

Test that the emergency account can make the repair.

Remote hands need an exact job

If remote recovery is blocked, onsite staff become your eyes and hands.

```
Location:   Frankfurt, rack R12
Equipment:  edge-a, port 2
Task:       verify label and photograph LEDs
Scope:      inspect only, no disconnect
Return:     timestamp, photos, label mismatch
```

For vendor escalation, add version, symptoms and logs.

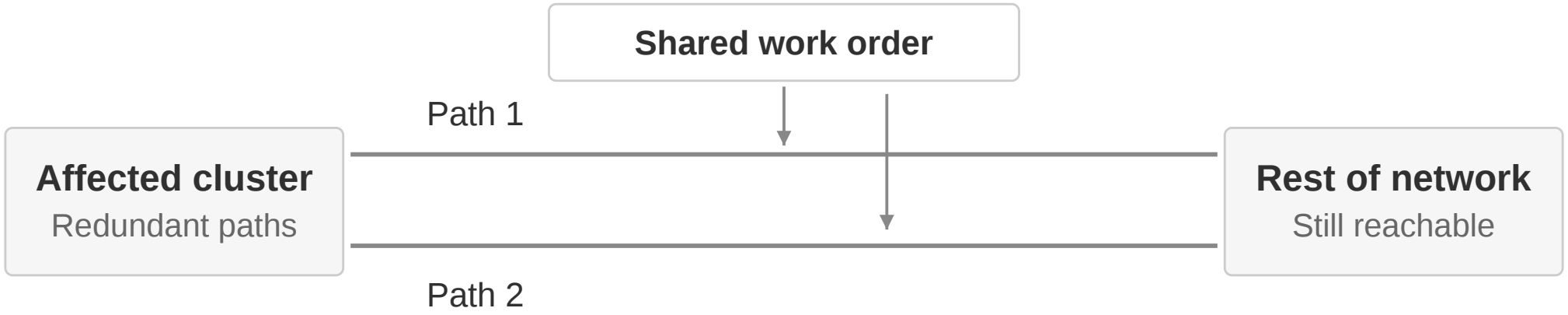
Google Cloud: incompatible optics

1 September 2026, us-central1

New router optics were incompatible with the fabric side.
The work crossed redundant routers before connectivity was checked.

Affected VMs lost connectivity in parts of us-central1-b and us-central1-f.

Google Cloud: one worklist defeated redundancy



1 September 2026: all affected paths were unplugged within **13 minutes**.
The worklist omitted router-by-router sequencing and verification.

Recovery: divert traffic, reinstall the original optics, verify links.

OVHcloud: undamaged did not mean available

Strasbourg, 10 March 2021. Fire began in SBG2 at 00:47 CET.

- SBG2 was destroyed. SBG1 lost four of its twelve rooms.
- SBG3 and SBG4 were undamaged, but powered down.
- Recovery involved site repairs, replacement servers and available backups.

Replacement capacity does not recreate lost data.

Same fire. Different recovery.

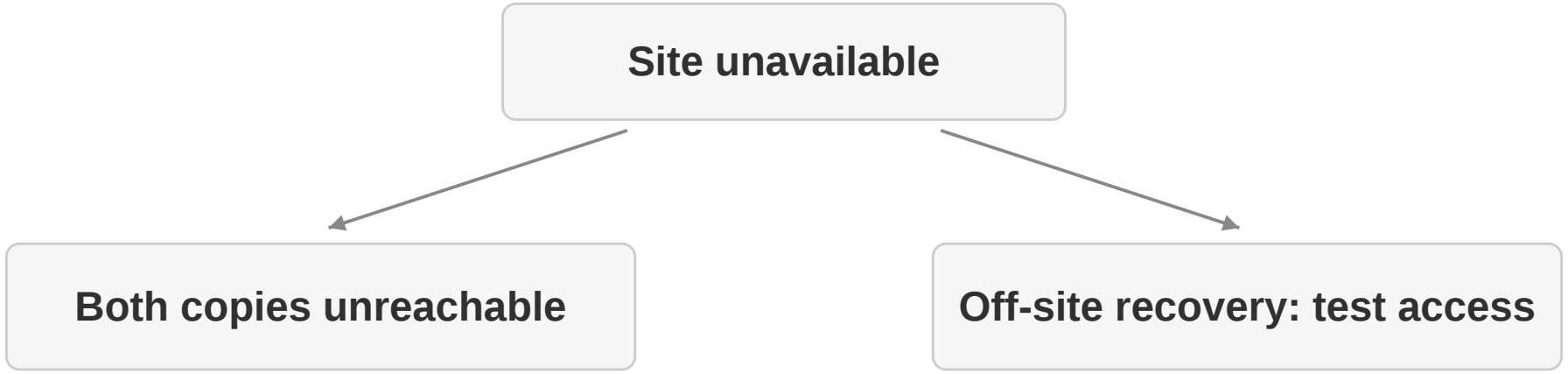
OVHcloud Strasbourg, 10 March 2021.

Service	What users lost	Recovery
Rust / Facepunch	Game data on 25 affected EU servers	Data unrecoverable
Lichess	24 hours of puzzle history	Backup from another data center

A replacement server does not bring back player progress.

Rust: reported impact Lichess: recovery report

Two copies, one failure domain

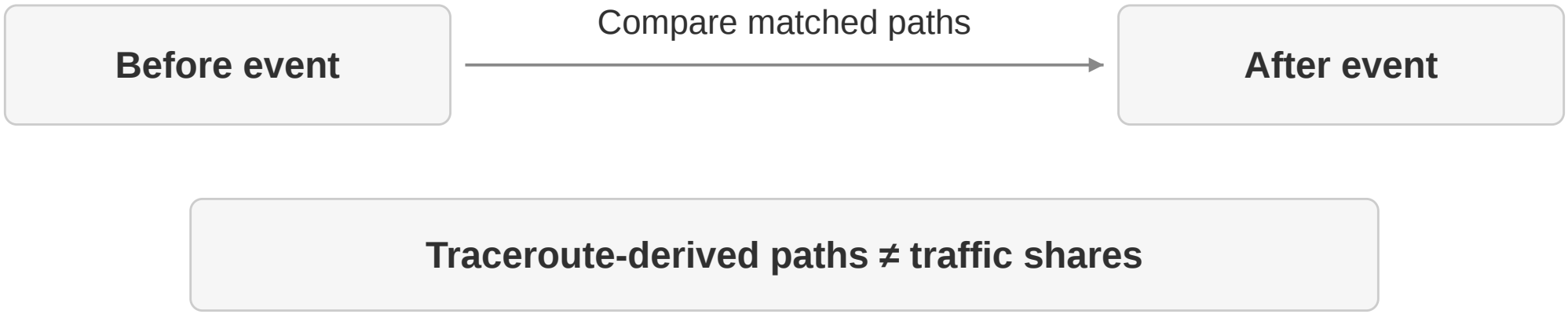


Copies do not remove a shared failure domain.

Exercise: assume both copies are inside the unavailable site.
Which recovery plan still works?

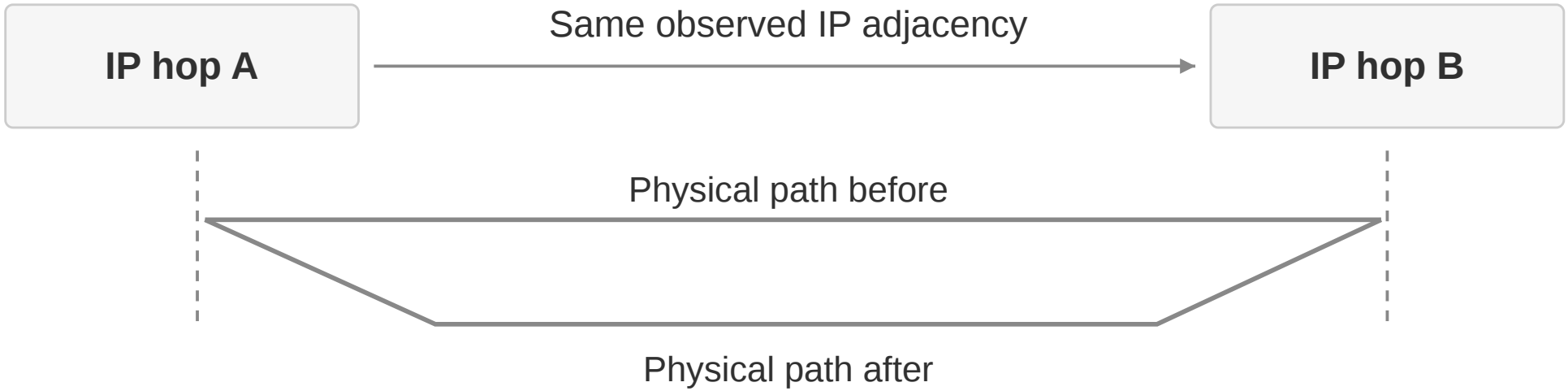
Baltic Sea: paths after the cuts

RIPE Atlas, November 2024: Germany–Finland paths.



A different physical boundary: the cable route. Compare matched observers.
This diagram shows the method, not traffic shares.

The IP path can look unchanged



Traceroute alone cannot identify a damaged cable.

Red Sea: damage is not attribution

24 February 2024: three cable systems were damaged.

Kentik's analysis supports an anchor-dragging explanation involving the *Rubymar*.

Keep the claims separate:

- network reachability and delay: observable
- physical cause: external evidence
- deliberate attack on cables: not established by the network data

Kentik: Red Sea (2024)

Red Sea: the cloud takes a detour.

September 2025: connectivity can continue while paths get slower.

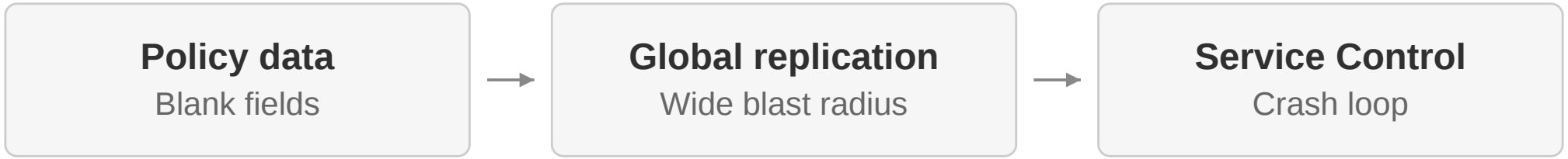


Triangulate evidence. An IP path cannot identify a damaged cable.

Which measurement separates **slower** from **unreachable**?

Google Cloud: a dormant bug goes global

12 June 2025. One policy update crosses regional boundaries.

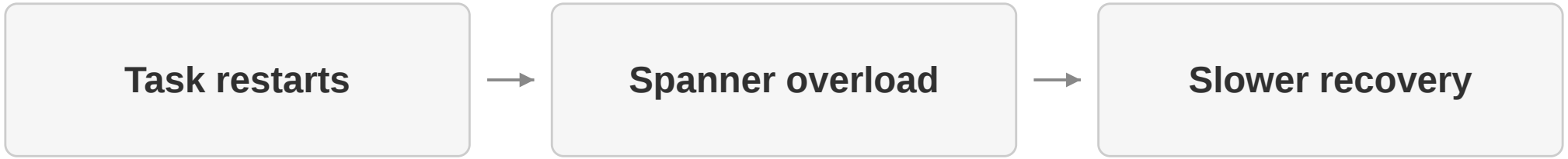


A null pointer crashed the API policy-checking process.

Service Control checks API policy. Blank fields triggered HTTP 503 errors.
Impact: API errors affected Cloud Storage, IAM and other products.

Google Cloud: recovery overloaded Spanner

12 June 2025. Disabling the faulty check stopped the crash loop.

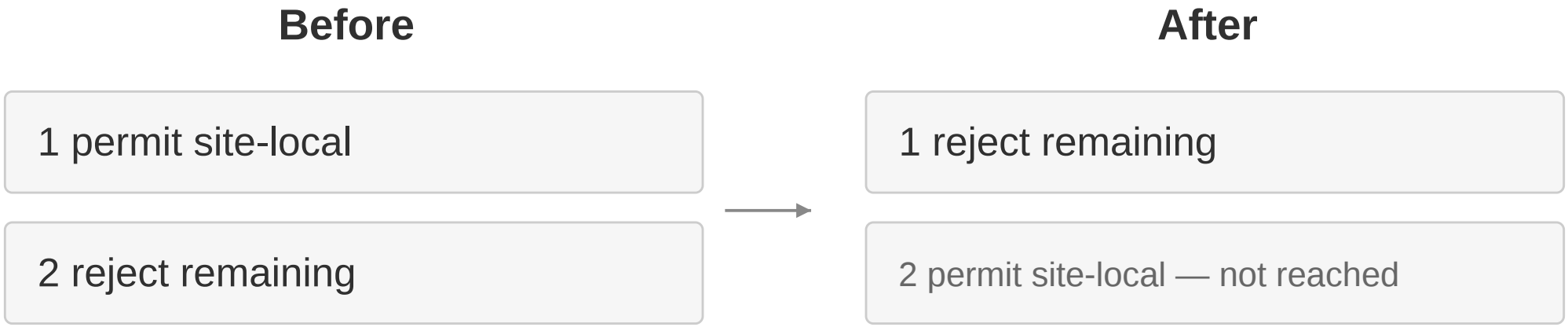


Missing randomized exponential backoff amplified the restart load.

In us-central1, Google throttled restarts and redirected database traffic.
Some customer monitoring failed too. The first status update took about an hour.

Cloudflare: policy order withdrew routes

21 June 2022. Simplified BGP export-policy order:



First matching rule wins. Later permission cannot undo rejection.

Site-local routes disappeared. Servers lost origin access.
Internal load balancing also failed.

The canary never exercised the new architecture

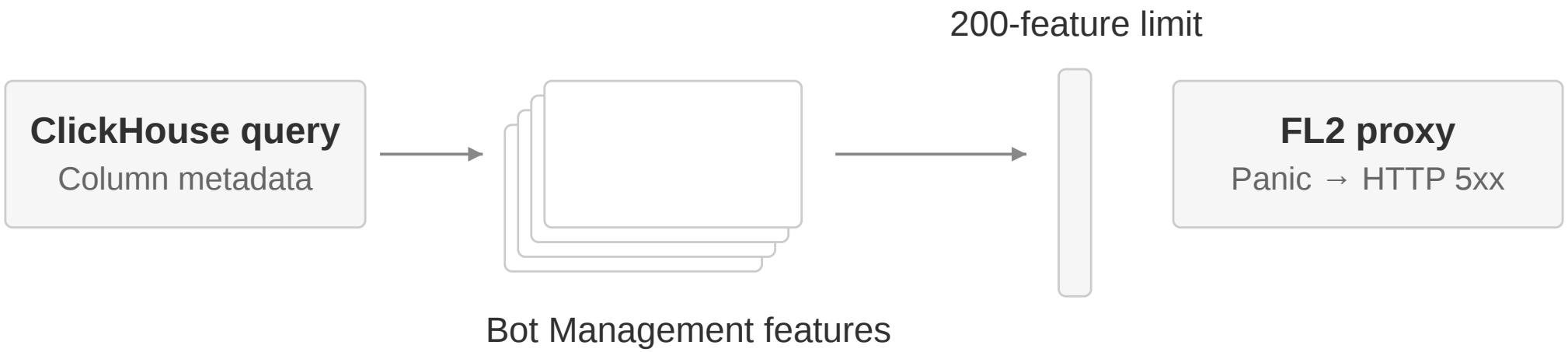
Cloudflare, 21 June 2022. Times in UTC.

- Early rollout stages used the older architecture.
- **06:27:** rollout reached 19 Multi-Colo PoP (MCP) sites.
- These sites handled about half of all requests.
- Backup access enabled rollback. The last revert finished at **07:42**.

Canary the architecture that runs the changed policy.

Cloudflare: metadata became a bad feature file

18 November 2025. A permissions change exposed more column metadata.



Without a database filter, duplicate metadata pushed the file past **200 features**.
Older proxies returned zero bot scores. Blocking depended on customer rules.

Cloudflare: stop distributing the bad file

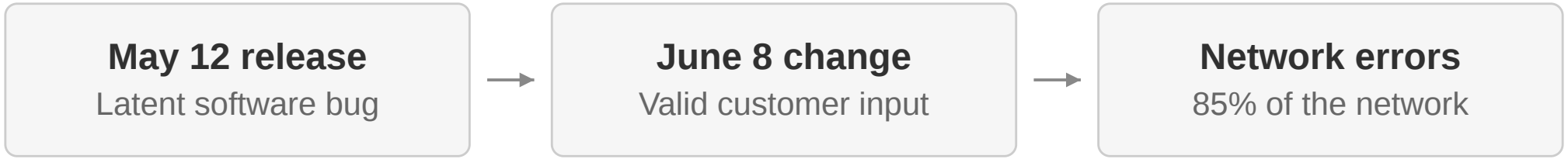
18 November 2025, UTC

Time	Recovery
14:24	Stop file generation and distribution
14:30	Restore most traffic with a known-good file
17:06	Recover remaining services

Websites returned 5xx. Turnstile blocked logins.
Recovery also required restarting the core proxy.

Fastly: valid input, latent bug

8 June 2021.



The input was valid. The software was not prepared for it.

Impact: CNN, The New York Times and Reddit became unavailable.
Fastly disabled the triggering configuration. The software fix followed later.

Fastly: detection is not recovery

8 June 2021. Times in UTC.



12:35: incident mitigated. **17:25:** permanent bug-fix rollout began.
Detection, mitigation and permanent repair are different clocks.

Fastly postmortem (2021)

Name the dependency.

Symptom	Hypothesis
Remote repair fails	Recovery shares the failed path
Redundant routers fail together	Shared procedure or site
All regions fail after an update	Global distribution
Green dashboard during impact	Missing or stale evidence

Your turn: redesign one boundary.

Pick one case. Draw its service path and recovery path.

Mark the shared dependency. Remove one dependency from recovery.
Propose a test that proves the revised path survives.

A second box is not a second failure domain.

From postmortem to early warning

Choose one failure pattern from today.

Which signal would have warned you?
How would you verify recovery?

14 October: monitoring and observability.

14 October 2026

Monitoring / Observability

In Operations, the customer had to tell us something was wrong.

How could the network have told us first?

What do we want to know?

Service	Network	Device
Availability	Reachability	Health
Latency	Loss	Capacity
Correct response	Routing state	Resource pressure

Choose the question before the tool.

An observation needs an expectation.

Monitoring: does behavior match the expectation?

Observability: can we explain the state from our observations?

Question → evidence → interpretation → action

Different signals answer different questions

Signal	Question
Metric	How much or how fast?
Event / log	What changed?
Flow record	Which traffic?
Packet capture	What was on the wire?
Active probe	Does this exchange work?

SNMP and gNMI deliver data. They are not signal types.

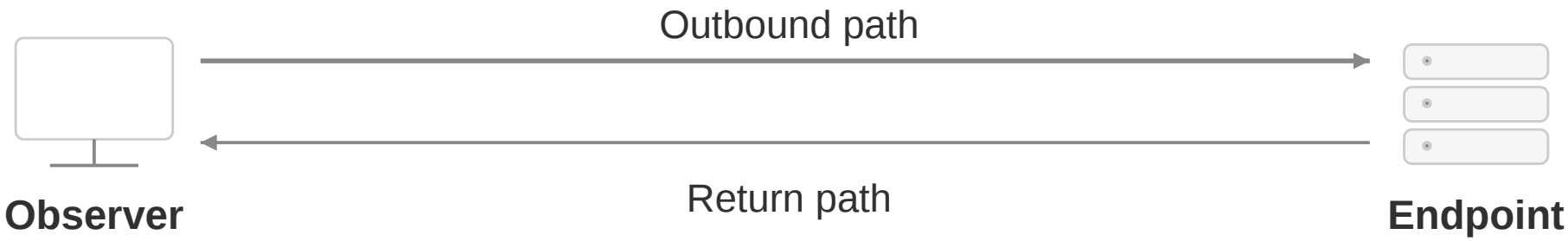
The monitoring brief

40 service-family targets: 20 services × IPv4/IPv6.
Two observation sources per target.

- Detect interruptions of **5 seconds or longer**.
- Stay within **20 HTTP requests/s**.
- Mark stale data **unknown within 15 seconds**.

Sizing exercise: this exceeds the small BOB1 probe set.

Delay and loss need endpoints



RTT contains both paths and endpoint processing.

One-way delay additionally needs synchronized clocks or bounded clock error.

Choose the probe from the question

Probe	Tells us
ICMP echo	IP reachability to a responding endpoint
TCP connect	transport handshake
HTTP request	application transaction
traceroute	responding hops along a path

A successful ping does not prove the web service works.

Define the probe before you run it

One BOB1 probe is:

```
source:          probe01.bob1.lagoontransit.test
target:          198.51.100.10
address family:  IPv4
protocol:        ICMP echo
reply wait:      1 s
success:         ping exits successfully
```

The lab exporter also runs DNS probes and repeats the checks continuously.

Two checks called "reachability" can still measure different things.

Budget the ReefNet probe round

For the smaller BOB1 probe set:

2 sources × 2 families × 3 probe types = 12 checks

A round every 5 seconds would need **2.4 checks/s**.

ReefNet waits for the round to finish, then pauses for 1 second.
What happens when a check times out?

Active probes: does this exchange work?

Observation	Source, target, family, protocol, time, result
Limit	Failure does not locate the fault
ReefNet	ICMP, DNS and HTTP over IPv4 and IPv6

DNS checks the expected answer. HTTP checks a 2xx response independently of DNS.

SNMP: what does the device report?

A manager queries an agent.

OID: object identifier.
MIB: Management Information Base, defining object meaning.

Keep the device, OID, row, value and poll time.
Short events can fall between polls.

Not provisioned in BOB1. Compare with gNMI.

Keep the object identity.

```
GET       request-id 41   ifOperStatus.17
RESPONSE  request-id 41   noError   INTEGER up(1)
```

The OID selects an object. `17` selects an interface row.
Check response status, device identity and the row's name/alias.

After restart or replacement, verify the mapping before joining counters.

Syslog: what changed?

Observation	Time, host, severity, app and message
Limit	Missing logs do not prove health
ReefNet	SR Linux → Alloy → Loki → Grafana

Keep source and receive times. Severity is the device’s judgment.

IPFIX: which traffic crossed here?

Observation	Flow keys, counts and time interval
Limit	Typical records omit payload. Sampling affects counts
ReefNet	Record exercise without a live exporter

A **template** defines the fields. Keep its exporter and observation domain.
Port 443 alone does not identify an application.

A template ID is not globally unique.

Illustrative ReefNet exporters, not provisioned in BOB1:

Exporter	Template 256 means
edge01	source, destination, bytes
edge02	source, destination, packets

Record from edge02: `203.0.113.10, 198.51.100.10, 8000`

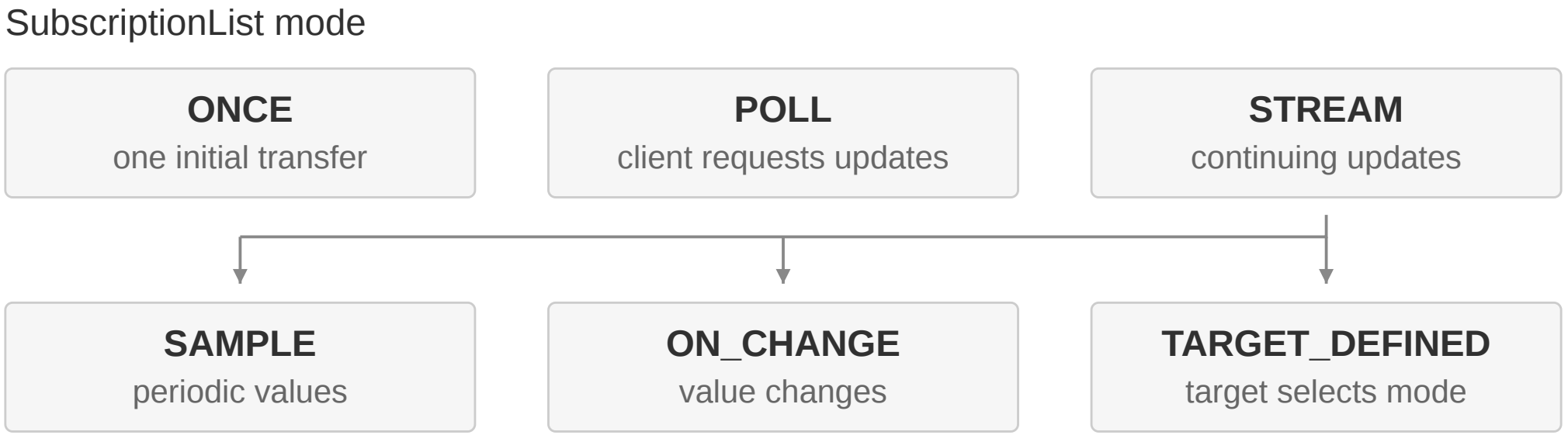
That is **8,000 packets**, not bytes.
What context must the collector retain to decode it?

Streaming state: what changed?

Observation	Target, keyed path, timestamp, update/deletion
Limit	Reconnect cannot restore every missed event
ReefNet	gNMIc samples SR Linux state and counters

The delivery mechanism does not define freshness.

Choose the subscription semantics.



SAMPLE is periodic. ON_CHANGE depends on target support.

Watch counters arrive through gNMI

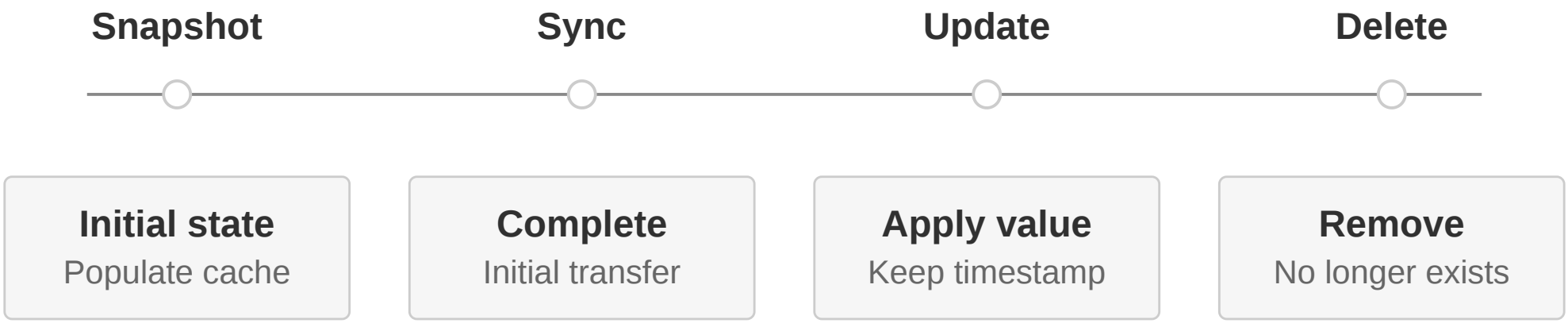
WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

```
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSrl1!' --skip-verify --encoding json_ietf \
  subscribe --path '/interface[name=ethernet-1/3]/statistics' \
  --stream-mode sample --sample-interval 5s
```

Compare with Grafana’s rate. **Ctrl+C** stops the subscription.

Build local state from a gNMI stream



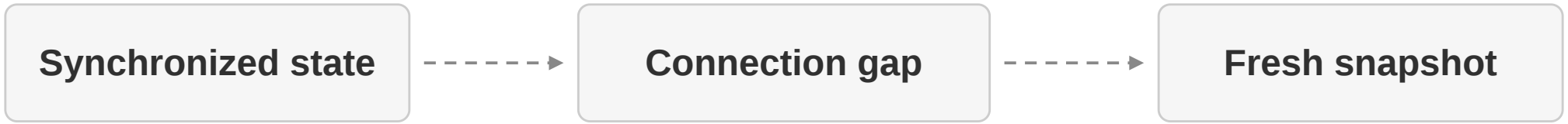
Sync completes the initial transfer. It does not guarantee future freshness.

Deletion is not zero

0	—	Deleted
Measured zero	**No fresh sample**	**Object removed**
Valid observation	State unknown	Update the model

These states need different storage, query and alert behavior.

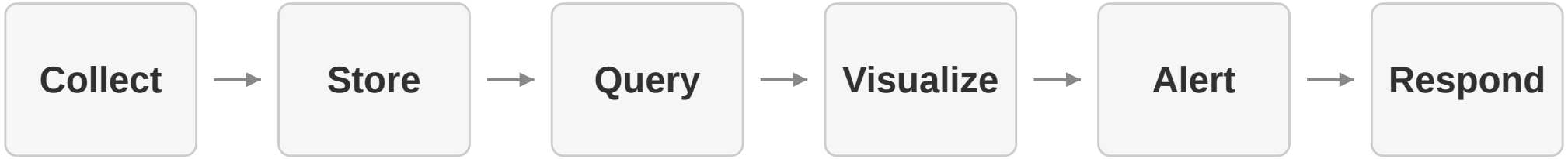
Reconnect cannot restore lost history.



Events during the gap may be irretrievable.

Rebuild current state after reconnect. Keep the observation gap visible.

Turn observations into a response.



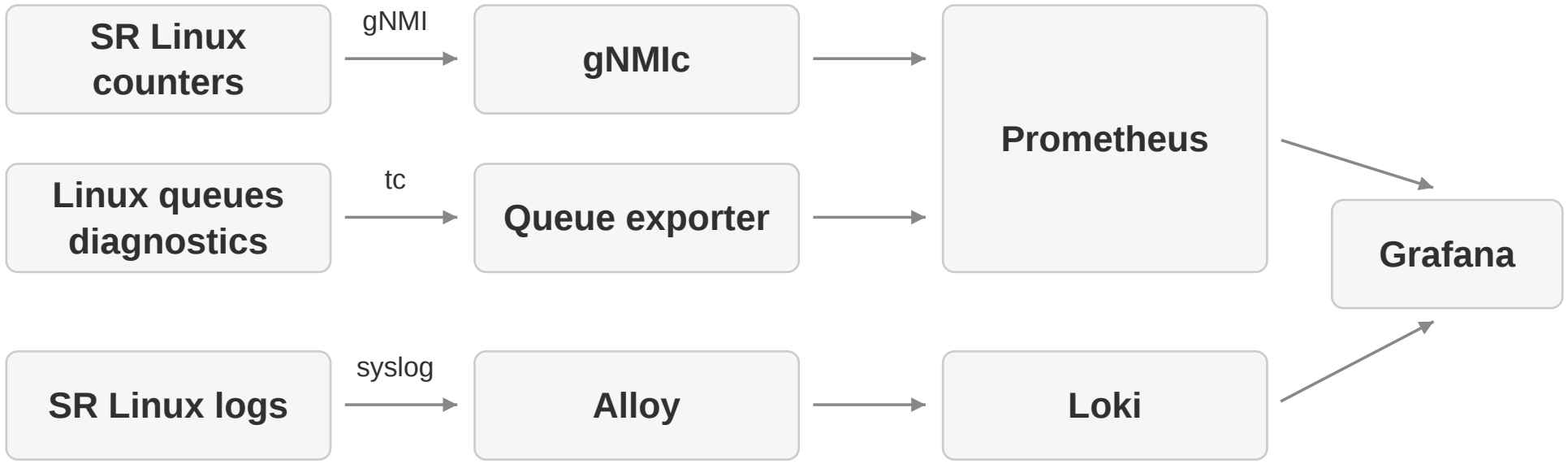
Every boundary can lose meaning, freshness or context.
Keep enough metadata to detect that loss.

Where should the collector live?

Placement	Main dependency
Central	WAN reachability
At the site	Site power and local collector
Several collectors	Coordination and duplicate handling

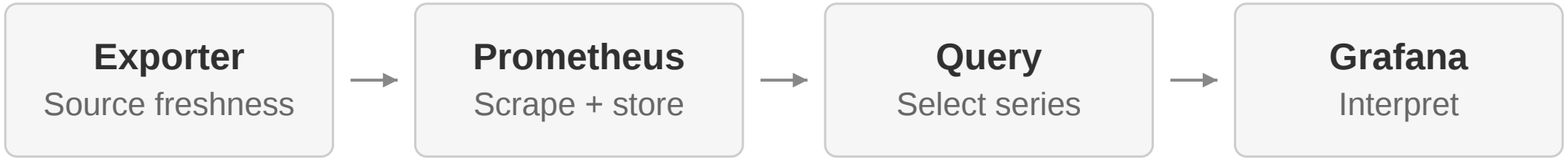
Test isolation. A second collector may share the first one’s failure domain.

Metrics and logs in ReefNet



Grafana queries metrics and logs. Each collection path can fail separately.

Prometheus stores time series.



A successful scrape can still return stale source data.

Try PromQL queries at `http://localhost:9090`.
Trace a suspicious panel back to its source timestamp.

Cardinality: count ReefNet's series

2	×	2	×	2
sources		IP families		probe types
Lagoon / Pacific		IPv4 / IPv6		ICMP / DNS

8 series for one metric

One target: data.oceanresearch.test

Prometheus: `count(reefnet_probe_success)` → **8**
Success, duration and timestamp together → **24 series**.

Would you add a request ID?

A proposed change to ReefNet's probe metric:

```
source="probe01.bob1.lagoontransit.test"
probe="icmp"
af="ipv4"
target="data.oceanresearch.test"
request_id="a-new-value-for-every-run"
```

What does `request_id` do to cardinality? Where should it go?

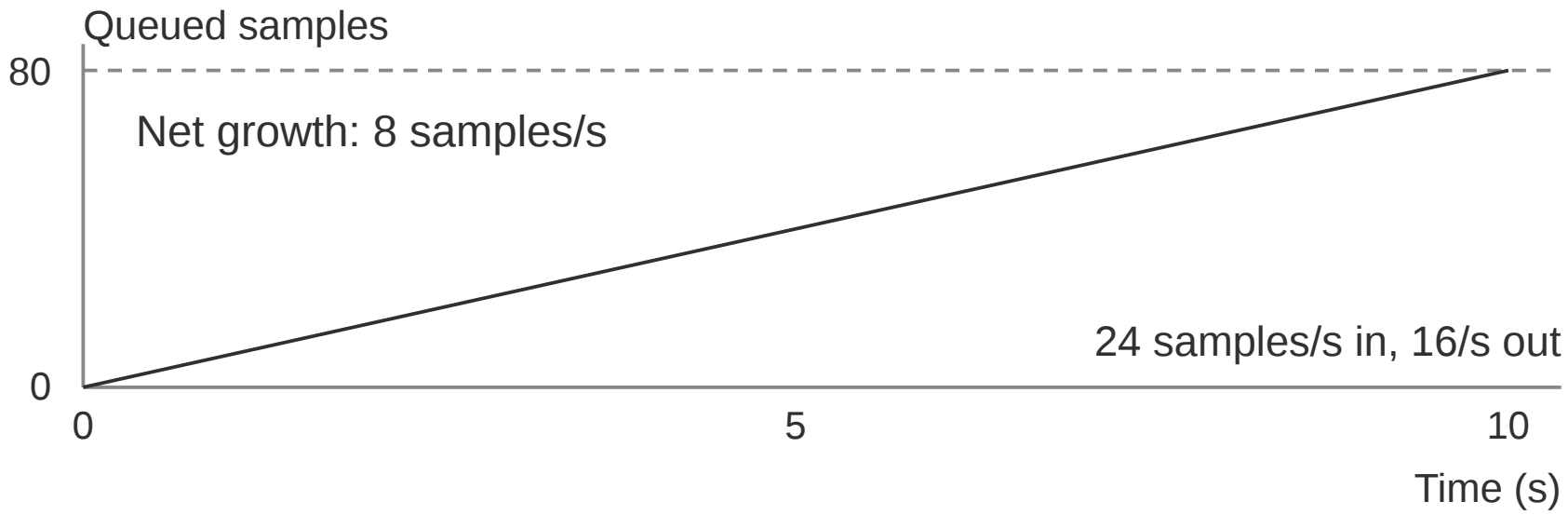
When would we need a queue?

Do independent consumers need to **replay the same observations**?

A retained event log may help. It also needs retention, lag monitoring and recovery.
For one dashboard, direct storage may suffice.

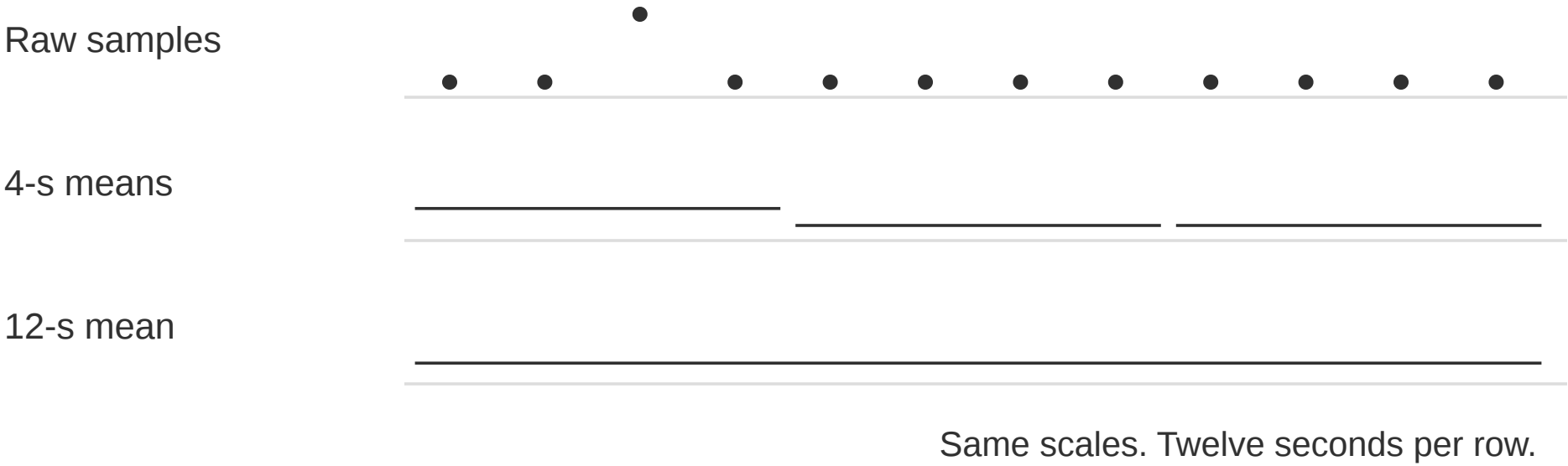
What if ReefNet adds a slow archive?

Design exercise: forward the probe samples to a remote archive.



The buffer buys ten seconds. It cannot fix the slow writer.

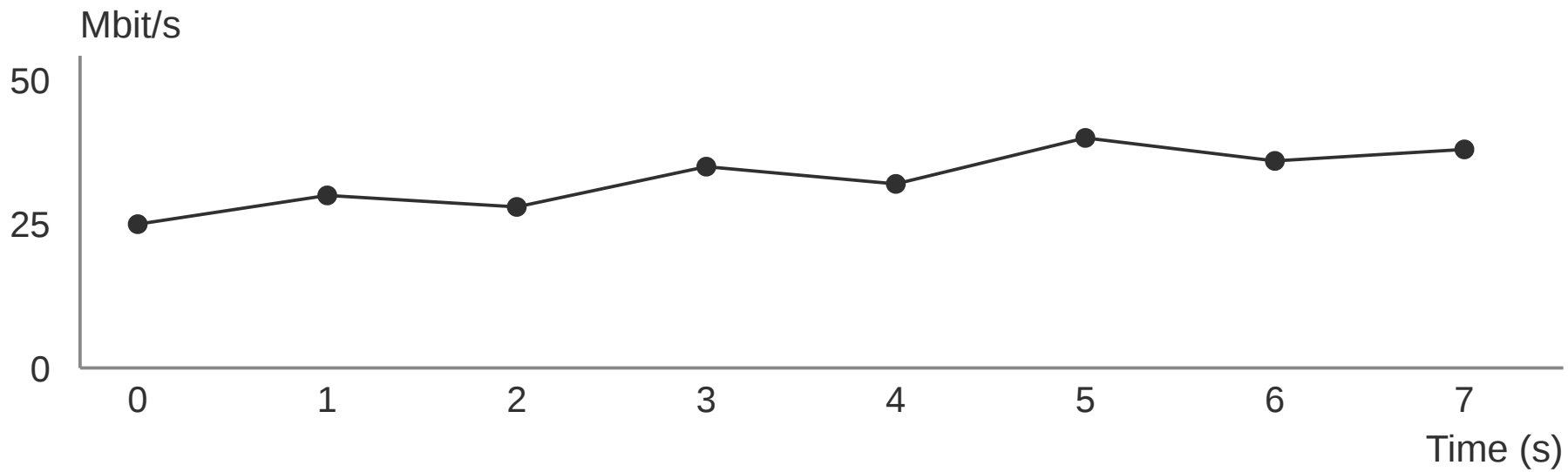
Retention changes what you can ask.



Same scales. Twelve seconds per row.

Aggregation saves space but can erase short failures.

One ReefNet interface, one series

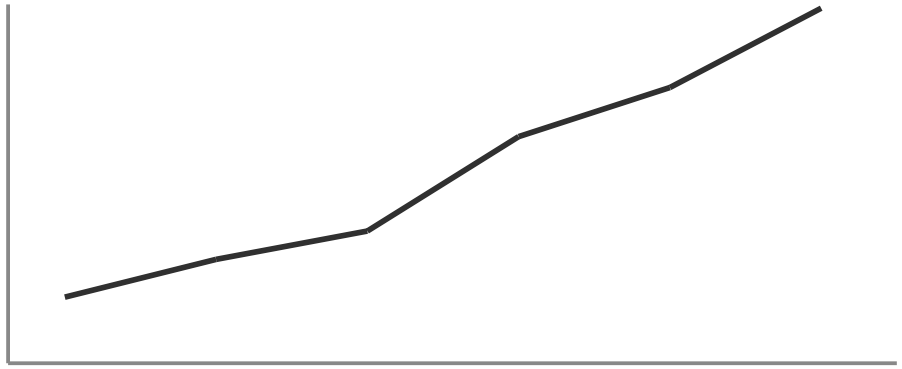


`edge01`, `ethernet-1/3`: traffic arriving from Lagoon Transit.

Each sample keeps that identity. The plotted values are illustrative.

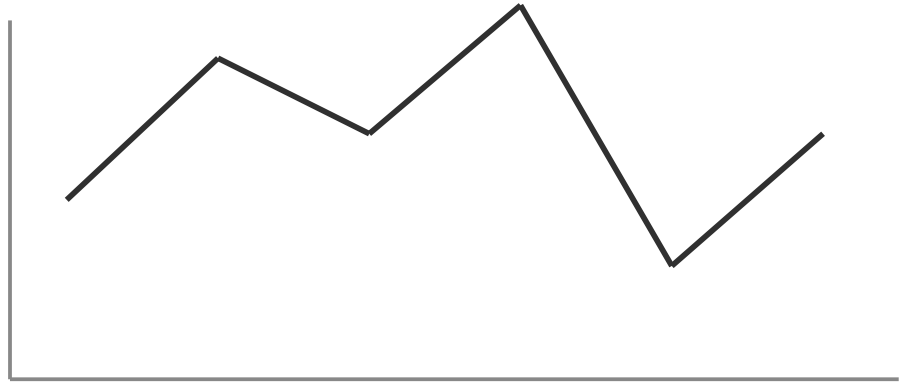
Counter or gauge?

Counter



Bytes on edge01 / ethernet-1/3

Gauge

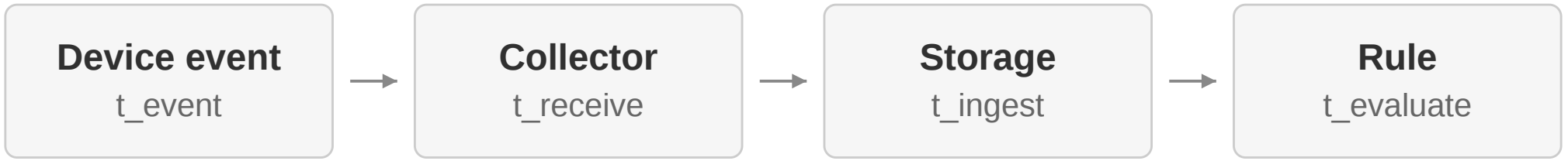


DNS-check duration from Lagoon

The byte counter accumulates. The probe duration can rise or fall.

Timestamps in the pipeline

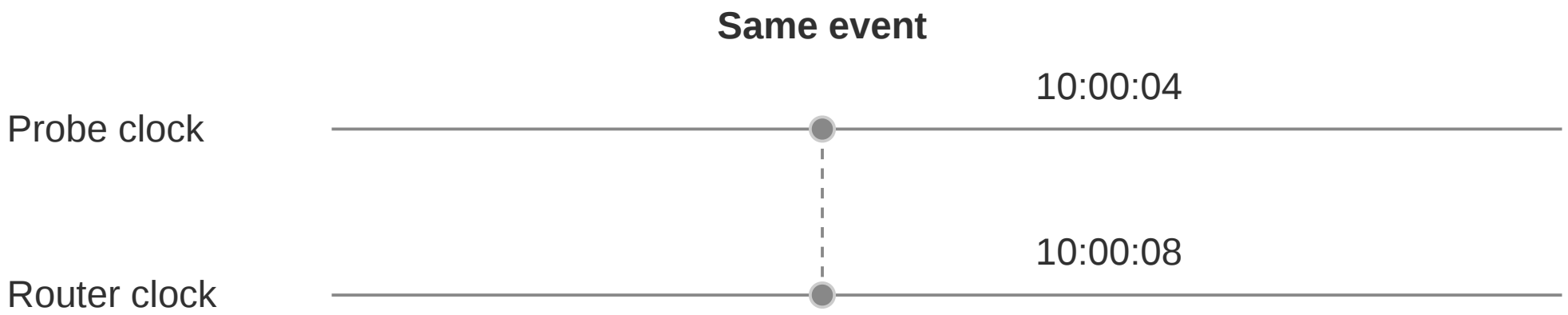
One event, several clocks.



Clock offset and transport delay are different quantities.

Subtract timestamps only after checking clock source and offset.

Clock error changes the story



Unsynchronized clocks distort the apparent event order.

Bound clock error before inferring event order.

Before subtracting counters

Check identity, timestamps and discontinuities.

A negative delta may mean **reset, wrap or bad data**.
Two values cannot reveal multiple wraps.

Invalid evidence becomes **unknown**, never a fabricated rate.

Turn ReefNet byte counters into bit/s

Use `in-octets` on `edge01`, interface `ethernet-1/3`.

$$rate = \frac{8(C_2 - C_1)}{t_2 - t_1}$$

Counter in **bytes**, time in **seconds**. Use a reset-free interval.

Compare the result with the **50 Mbit/s Lagoon handoff**.

Calculate the Lagoon ingress rate

Illustrative `in-octets` on `edge01`, interface `ethernet-1/3`.

Time	Byte counter	Reset?
10:00:00	1,000,000	Initial sample
10:00:10	32,250,000	No
10:00:20	20,000	Yes
10:00:32	37,520,000	No

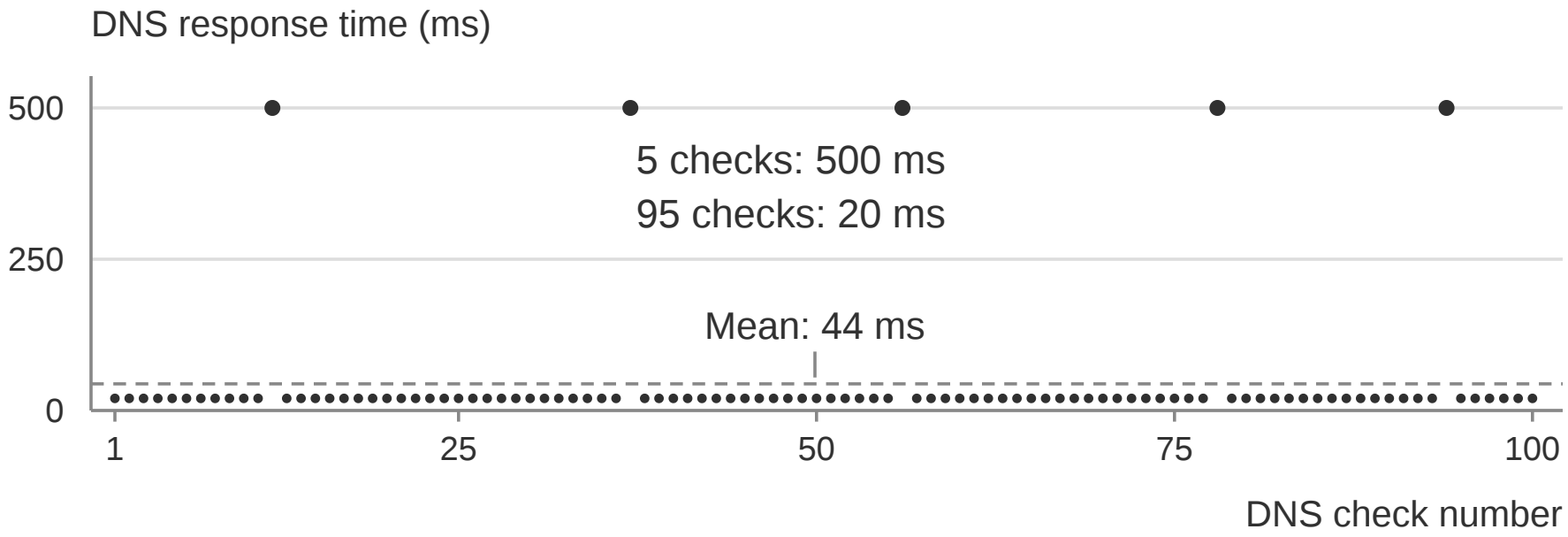
Four minutes: calculate valid rates. Compare them with the 50 Mbit/s handoff.

25 Mbit/s on a 50 Mbit/s handoff

Interval	Rate
00–10 s	8 × 31,250,000 / 10 = **25 Mbit/s**
10–20 s	Reset: **exclude**
20–32 s	8 × 37,500,000 / 12 = **25 Mbit/s**

Both valid intervals use **50%** of handoff capacity.

What Lagoon sees when it probes DNS



100 illustrative DNS checks to Ocean Research. Start with the individual waits. Would the **44 ms mean** explain the five long waits?

Read the distribution

View	What it tells us
Median	Middle observation
p95	Threshold covering at least 95%
Histogram	Counts within ranges
Cumulative distribution	Fraction at or below each value

A percentile is not an “average slow request”.

Percentiles do not average

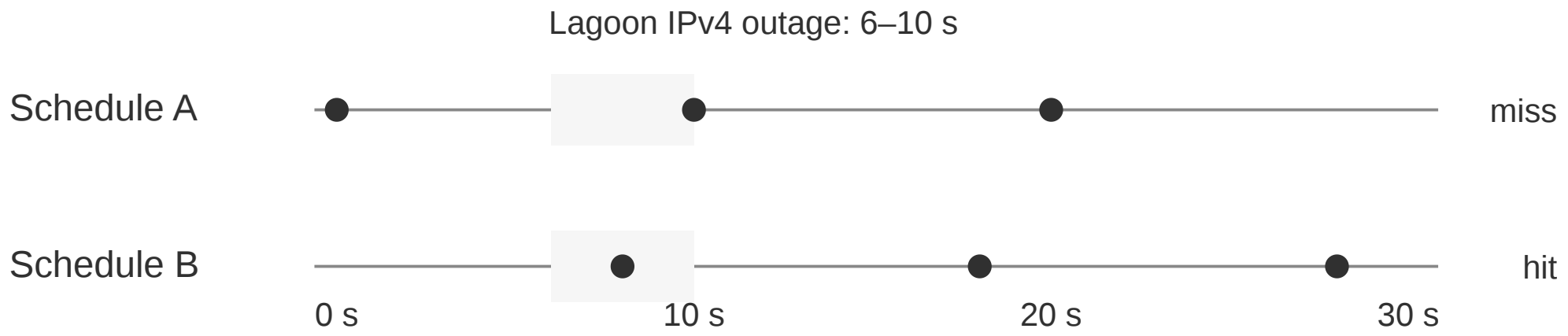
A separate DNS example for `data.oceanresearch.test`. Nearest-rank p95:

Source	Checks	Duration of each check
Lagoon	100	20 ms
Pacific	1	1,000 ms

Average of the two p95s: **510 ms**.
Combined observations: **p95 = 20 ms**.

Combine observations first, then calculate the percentile.

Four seconds can disappear



Same 10 s interval. Different onset phase.

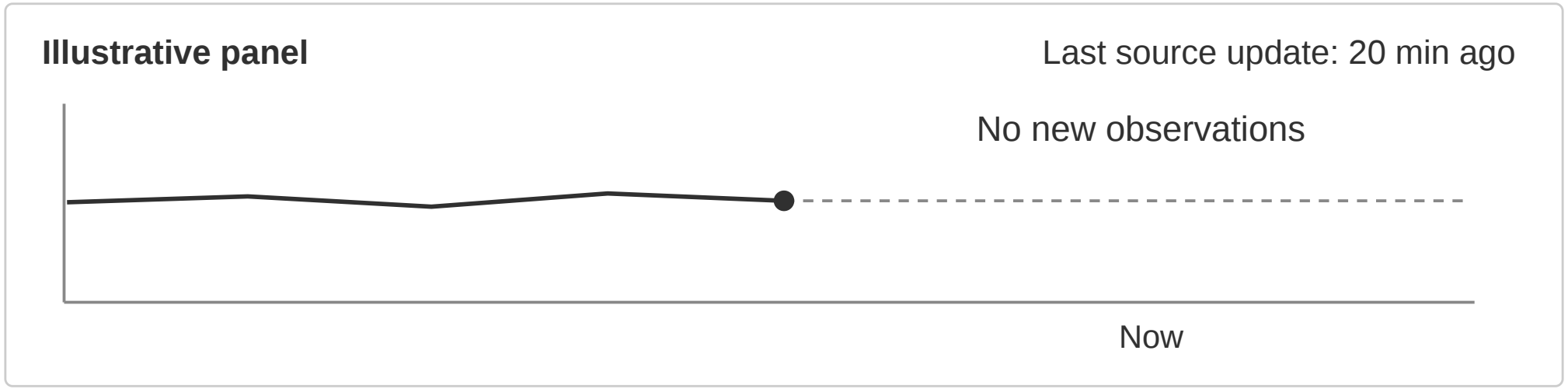
Imagine a short Lagoon IPv4 outage. A healthy sample can miss it.

Sampling leaves gaps

A fault can begin and end between samples.

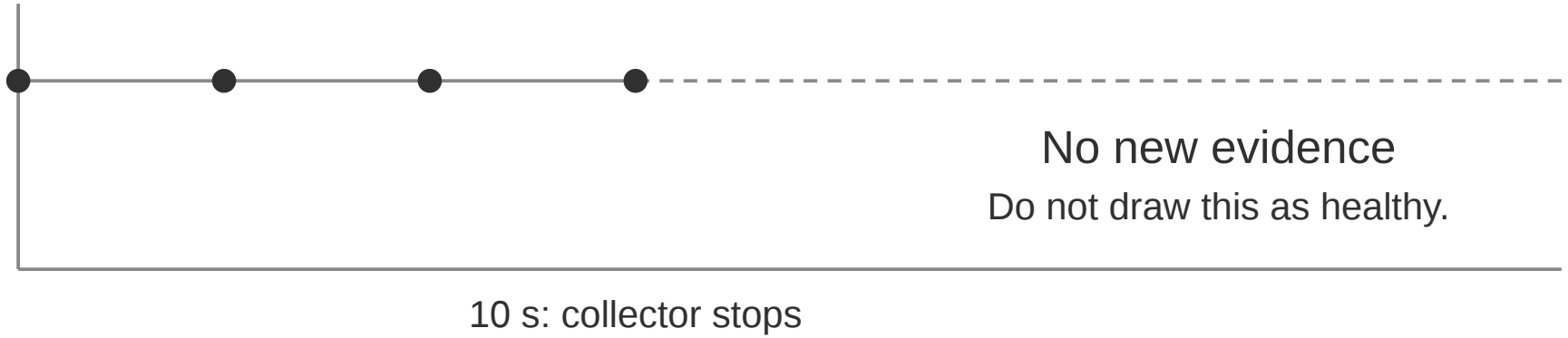
Interval and onset phase determine whether we see it.
Streaming sampled values still leaves gaps.

Up can still mean stale.



Watch source age, even when the scrape succeeds.

Missing data on a dashboard



When should the panel stop looking healthy?

Monitor the monitoring

Component	Watch
Collector	Last valid observation
Broker / consumer	Buffer use, progress and event age
Rule engine	Errors and evaluation time
Notification path	Delivery result

Check the central monitoring service from outside.

Choose a dashboard

Question	BOB1 dashboard
Network status and topology	Network Overview
Reachability and response time	Service Health
Sessions, routes and updates	BGP
Core adjacencies and SPF activity	OSPF
Port state and errors	Devices & Interfaces
Rates and utilization	Traffic & Capacity
Collection and freshness	Telemetry Health

What is Grafana telling us?

At **25 Mbit/s IPv4 traffic**, open `http://localhost:3000`.
Choose **ReefNet / BOB1 → Network Overview**.

- What is measured, and where?
- Which units and time window?
- How fresh is it?

Predict what changes when we send more traffic.

Turn up the traffic

In WSL, choose one load:

```
make traffic-40mbit
make traffic-10mbit
make traffic-25mbit
```

Allow 30 seconds at each level.
Watch the interface counter and its Grafana rate.

Which changes first? Where does the delay come from?

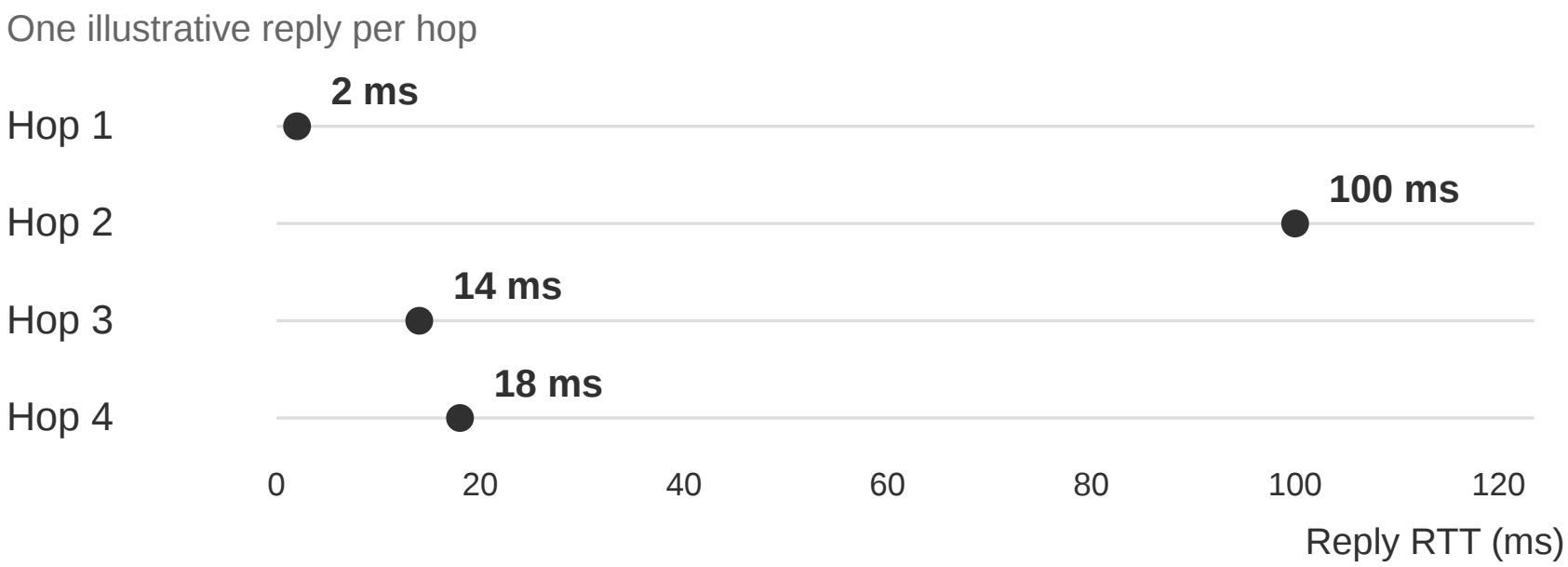
Same symptom, different causes

Lagoon IPv4 fails in both cases.

Evidence	Export rejected	Session down
BGP session	Established	Not established
Customer prefix exported	No	No

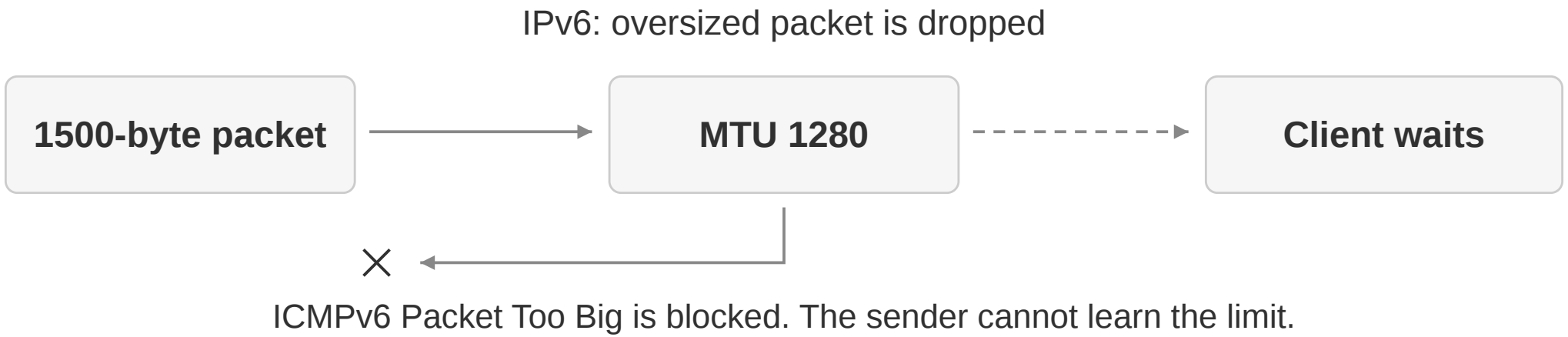
Session state distinguishes these hypotheses.

Traceroute: read replies carefully



Hop 2 replies slowly. It does not delay all forwarded packets by 100 ms.

Small ping, stalled download.



IPv6 routers do not fragment transit packets. **ICMPv6 Packet Too Big** drives PMTUD.

One link goes down. What follows?

In WSL: `make fault-link`

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show interface ethernet-1/4 detail
show network-instance default protocols ospf neighbor
```

Check **Service Health** and **Devices & Interfaces** in Grafana.

Did the customer notice? Explain the path that remains.

Admin state and oper state

The fault just injected disables `ethernet-1/4` on ReefNet Edge 01:

```
/interface[name=ethernet-1/4]/admin-state   "disable"
/interface[name=ethernet-1/4]/oper-state    "down"
```

Admin: configured intent. **Oper:** observed behavior.

Prometheus exposes `reefnet_interface_admin_state` and `reefnet_interface_oper_state`. Read them with their timestamps.

Restore the link and watch convergence

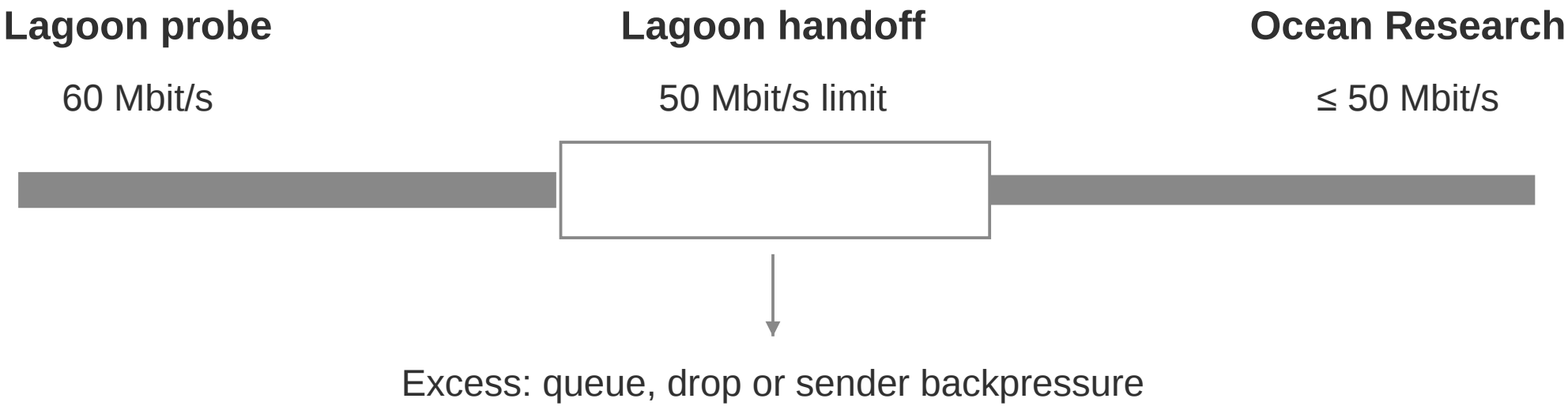
In the WSL repository:

```
make clear-link
```

Verify admin enable, oper up and the OSPF adjacency.
Correlate event time with external reachability.

The Lagoon-to-customer flow does not need `core-b`.
Its service check can remain green while the topology changes.

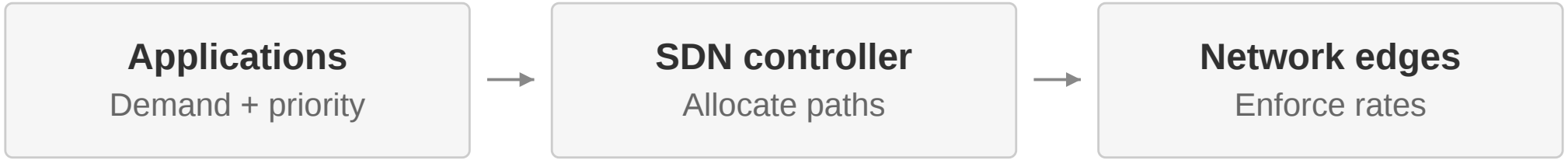
Capacity is not throughput



A faster customer handoff does not remove the Lagoon bottleneck.

Google B4: busy by design

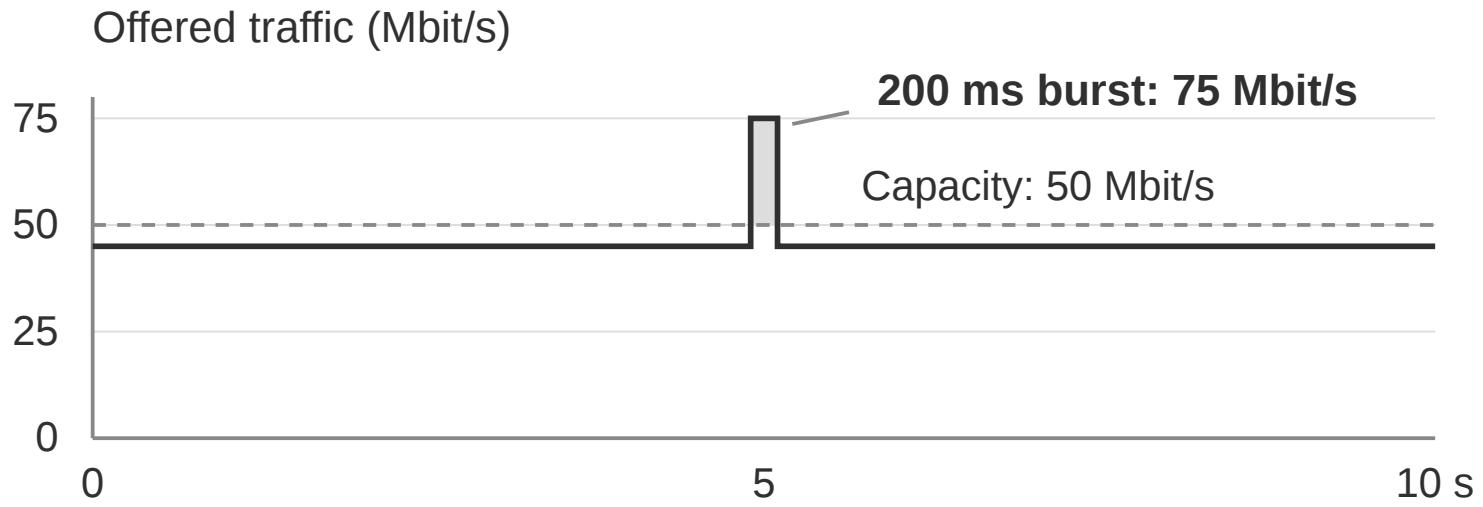
Google’s private WAN ran many links **near 100% utilization**.



Capacity is allocated across paths as demand and failures change.

Google controls the senders. Bulk transfers yield to higher priorities.

90% average. Still dropping packets.



Dashboard

≈90%

mean output
over 10 s

Synthetic Lagoon example. The brief peak is lost in the average.

A full queue drops packets. Check **drops and peaks**, not only averages.

Where do the packets go?

What happens when offered traffic exceeds the **50 Mbit/s** handoff?

In WSL:

```
make monitoring
make task
```

75 minutes: compare load, received traffic and new discards.
Then try link and routing faults and a short burst.

Exercise overview

How much traffic reached the customer?

WSL `make enter NODE=edge01.bob1.lagoontransit.test`

Router CLI: physical interface counters

```
info from state interface ethernet-1/1 statistics
```

WSL after quit `make enter NODE=service01.bob1.oceanresearch.test`

Inside the receiver: Mbit/s and loss

```
tail -n 15 /tmp/iperf-server.log
```

Compare new egress discards with receiver loss.
Use `exit`, then in WSL: `make traffic-10mbit`.

Keep the simulator out of the result

75 Mbit/s, 1,200-byte UDP datagrams:

≈ 7,800 packets/s < 10,000 packets/s container limit

The **50 Mbit/s handoff** should limit first.
Linux queue drops may mirror SR Linux discards. Do not add them.

Blink and you miss it?

In WSL, stop background traffic and send a five-second burst:

```
make traffic-stop
make traffic-burst-40mbit
```

The burst stays below the 50 Mbit/s cap.
What does it look like in a graph with a long rate window?

Can you recover its height and duration?
When finished: `make traffic-25mbit`

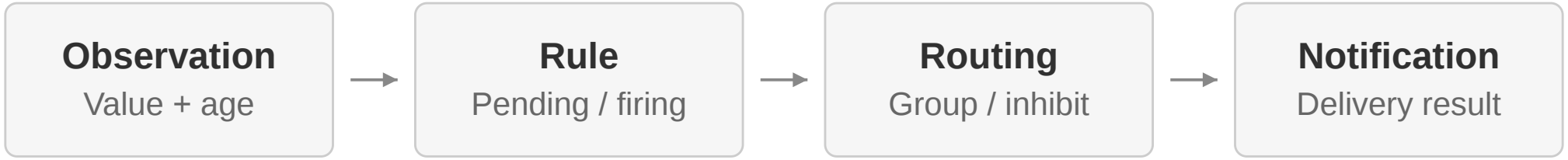
Several signals, one incident.



One controlled link fault, several observations.

Illustrative timestamps. Check clock offsets before inferring order.

The trigger is not the notification.



The rule fires, but nobody is paged. Which boundary failed?

An alert rule from the lab metric

Illustrative Prometheus rule, using the actual BOB1 metric:

```
- alert: LagoonIPv4ProbeFailed
  expr: |
    reefnet_probe_success{
      source="probe01.bob1.lagoontransit.test",
      probe="icmp", af="ipv4"
    } == 0
  for: 30s
```

The condition must stay active across evaluations for 30 seconds.
This sustained-failure rule does **not** meet our 5-second detection brief.

Alert on stale observations

```
time() - reefnet_probe_last_run_timestamp_seconds > 15
```

This detects old exported results.
Also check missing series and scrape health.

Use the expected target set to detect disappearing sources.

Flapping and notification noise

Flapping: repeated state changes.

Mechanism	Purpose
Deduplication	Skip duplicate notifications
Inhibition	Mute notifications while a related alert fires
Grouping	Combine related notifications

A firing delay can prevent short-lived conditions from becoming alerts.

Precision and recall

	Fault	No fault
Alert	**90 true positives**	**99 false positives**
No alert	**10 false negatives**	**801 true negatives**

Recall = 90%. Precision = 47.6%.

More than half the alerts in this synthetic example are false.

SLI and SLO

Service Level Indicator (SLI): measured service behavior

Service Level Objective (SLO): target over a defined window

$$SLI = \frac{successful\ requests}{eligible\ requests}$$

Define how missing records are handled before calculating the SLI.

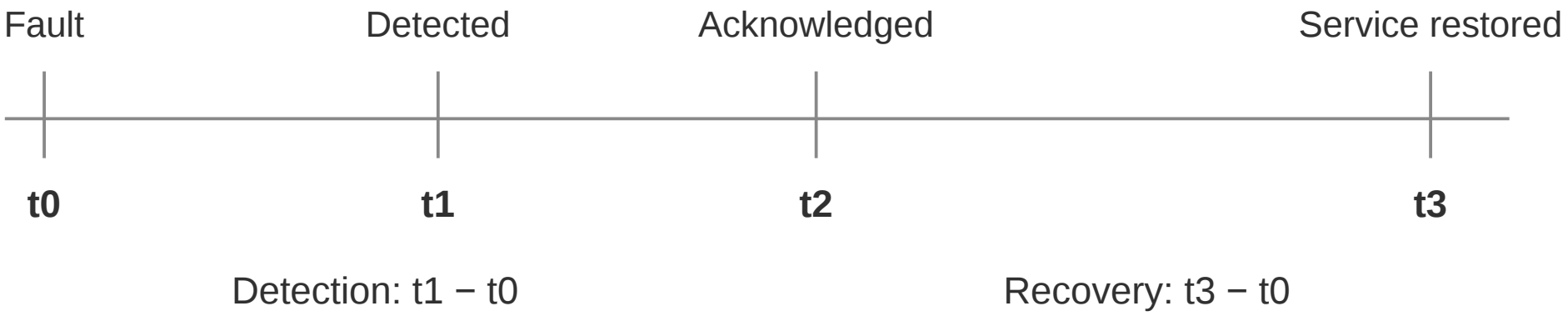
An error budget for the Lagoon probe

IPv4 ICMP from Lagoon to Ocean Research:
99.9% SLO across 10,000 completed checks.

$$10000 \times (1 - 0.999) = 10 \text{ allowed failures}$$

Six failures leave **four**. Report missing checks separately.

Detection and recovery are different clocks.



Name the endpoints before reporting a recovery duration.

Rehearse a response.

Five-minute tabletop during a change:

Vantage point	Service result
Lagoon, IPv4	Fails
Pacific, IPv4	Works

Name the first observation, incident owner and backout trigger.
Explain why that observation distinguishes the leading hypotheses.

Test the monitoring brief

Requirement	Test
Detect 5 s interruptions	Vary onset phase
≤ 20 requests/s	Count requests and retries
Unknown within 15 s	Stop collection
Useful diagnosis	Compare route loss and drops

Sketch a design for the **40-target brief**.

What would you tell the customer?

Was it congestion, a missing route or missing data?
Choose the measurements that explain your answer.

Note your observations, then stop the lab:

```
make traffic-stop
make down
```

Use `make monitoring` to start a fresh experiment on a configured lab machine.

30 October 2026

Research questions and sources

Monitoring gave us a limitation: short events can disappear between samples.

Now we turn that into a question we can actually test.

Leave with a testable research brief

1. State a question that a comparison can answer.
2. Find prior work and evidence that fit the question.
3. Choose a baseline and expose the biggest feasibility risk.

Our running example: **short outages missed between probes**.

Some project ideas

Area	Possible comparison
Monitoring	Polling vs streaming for short failures
Digital twins	Emulation vs modeling before deployment
Automation	Automated vs manual emergency repair
AI for networking	LLM troubleshooting vs a runbook

Bring your own question if you can evaluate it.

From a problem to a research question

At equal cost, which schedule catches short outages better?

Compare	Periodic vs randomized
Keep equal	Probe budget, workload, endpoint
Measure	Detection fraction and delay
Bound the claim	Tested fault durations and conditions

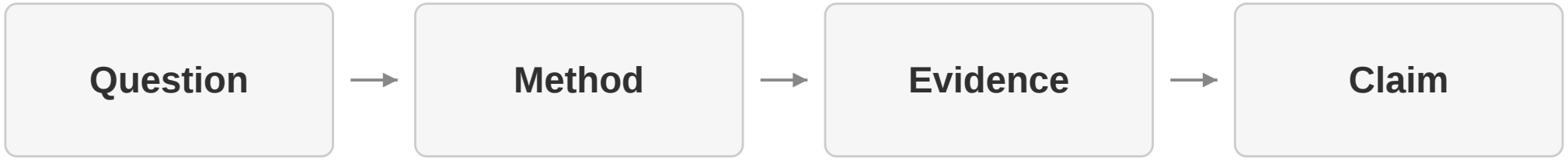
Write a measurable question.

Name the **factor**, **outcome** and **conditions**.

> At equal probe cost, which schedule detects more 1–10 s outages?

Define the schedules, fault distribution and detection criterion.
The comparison must allow your preferred method to lose.

What makes a scientific argument?



The claim cannot be stronger than its weakest evidence boundary.

Your contribution is the finding this evidence supports.

What kind of paper will you write?

Approach	Contribution
Measurement	Describe behavior
Comparison	Evaluate methods fairly
Reproduction + extension	Check and extend a result
System study	Build or investigate a system

Every approach needs evidence for a limited claim.

Task: challenge the claim

> "Randomized polling detects outages faster."

Two minutes:

1. Name a result that would contradict this.
2. Specify outage durations, baseline and measured outcome.
3. Choose a success criterion that allows the method to lose.

Plan the measurement from the claim

Claim	Metric	Evidence
faster detection	detection delay	repeated known faults
lower load	bytes/s or requests/s	collector + target
fewer misses	detection rate	known injected events

If the measurement cannot test the claim, change the claim or the experiment.

Observation or intervention?

Observational study	Controlled experiment
polling gaps coincide with missed events	deliberately vary the polling schedule
shows association	tests an intervention

A **confounder** changes both condition and outcome.

Injecting a fault is not enough. The comparison still has to be controlled.

Find the closest comparison.

Search by **problem, mechanism and method**. Follow citations.

Compare assumptions, baselines and evidence.
Record your queries and selection criteria.

“Nobody has done this before” is not a literature review.

Research venues close to this course

Venue	Typical contribution
IEEE/IFIP NOMS	operating, managing and automating networks
CNSM	management systems, telemetry and orchestration
ACM IMC	measuring and understanding Internet behavior

Start here for literature on **management, monitoring and measurement**.

Broader networking research

Venue	Focus
ACM SIGCOMM	Architectures and systems
USENIX NSDI	Implemented networked systems
IEEE INFOCOM	Protocols and performance
ACM CoNEXT	Experimental networking
IFIP Networking	Protocols and systems
IEEE GLOBECOM / ICC	Communications and networking

Who develops Internet technology?

IETF

Working Groups engineer interoperable Internet protocols and standards.

IRTF

Research Groups study long-term questions about Internet technology.

Where do operators compare notes?

RIPE Meetings
Routing, addressing, measurement and Internet coordination

DENOG and NANOG
Backbone operations, routing, peering and incidents

NAF and AutoCon
Network automation, orchestration and operational tooling

Read in three passes.

Pass	Look for
Relevance	Question and main result
Method	Signal, baseline and experimental unit
Challenge	Missing evidence and transfer limits

Record **one supported claim, one limitation and one next question**.

Read the claim before the graph.

Trinocular probes **IPv4 /24 blocks**, not applications.

Check	Ask
Scope	Which blocks and vantage points?
Method	Which outage definition and budget?
Validation	Which independent evidence?

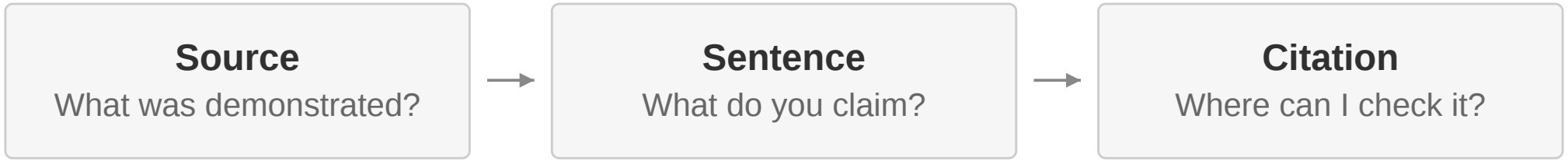
A reachable block does not prove application health.

Match the source to the claim.

Claim	Evidence source
Required protocol behavior	Specification and exact section
Implemented feature	Versioned code or official documentation
Measured improvement	Paper and evaluation data
Production failure	Attributed operator report

For RFCs: check status, updates, errata and normative language.

Does the citation support the sentence?



Match the sentence to the evidence in the source, not merely to its topic.

Paraphrase the idea, not the word order

Approach	Assessment
Replace a few words	Still follows the original
Explain from your notes	Paraphrase, with attribution
Copy exact wording	Quote, with location

A citation alone does not mark a quotation.

A DOI is not evidence.

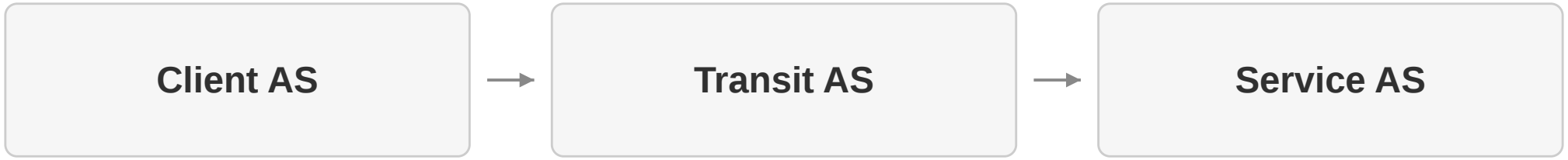
Read the result. Then check the claim.

Make evidence traceable.

Artifact	Record
Software	Version or commit and repository
Dataset	Release, identifier, access conditions
Documentation	Exact page, version and section
Figure	Data and units, or source and adaptation

Verify imported reference metadata. Mark synthetic examples.
Citation does not replace reuse permission.

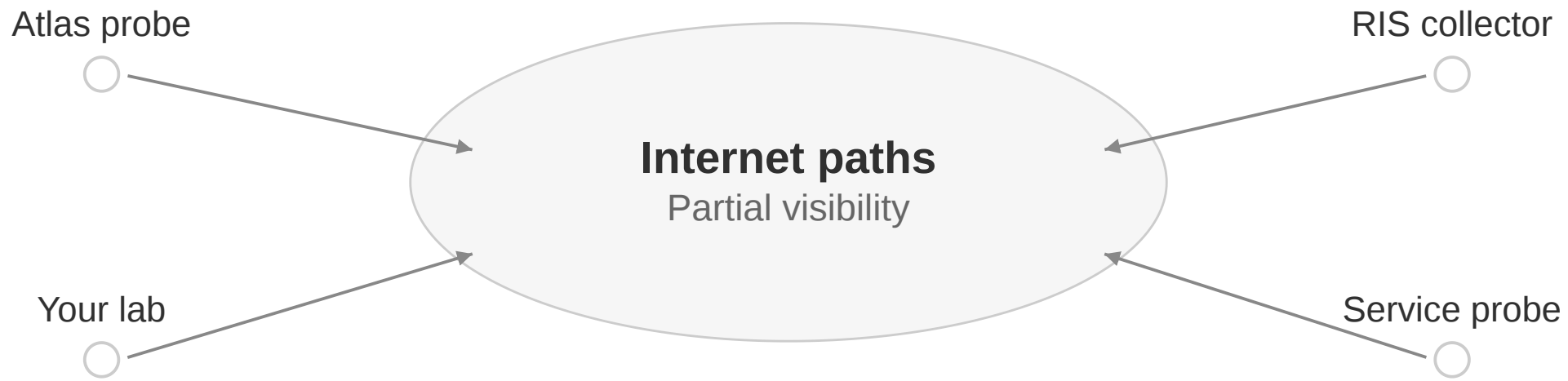
Can public data answer your question?



Each observer sees only part of this path.

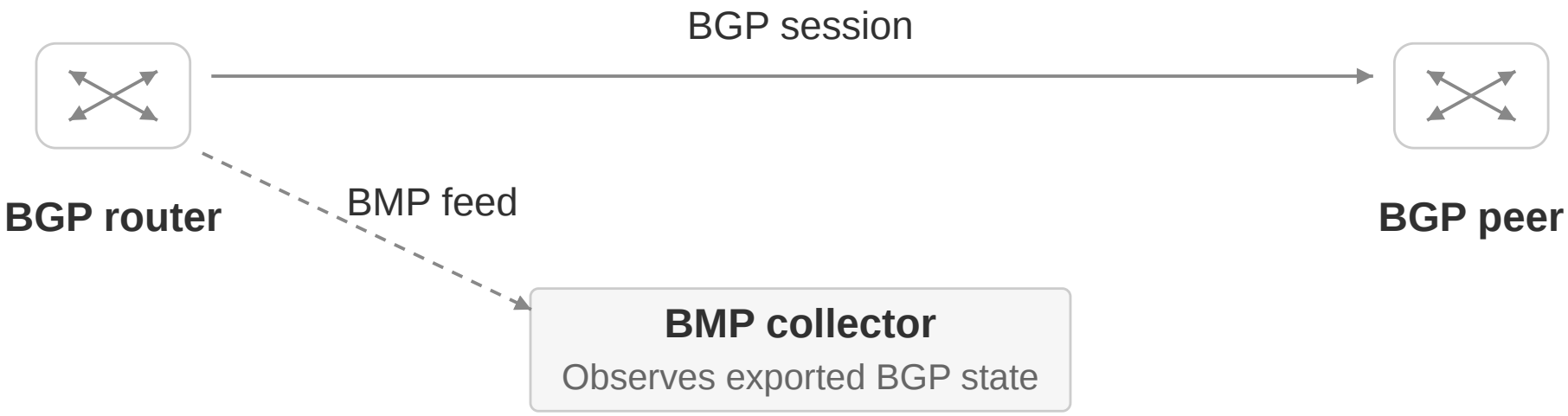
If your question extends beyond ReefNet, consider public observations.
Their coverage and resolution must fit the claim.

Public data gives us another vantage point



RIPE Atlas: active probes. **RIS:** public BGP observations.
Neither covers every Internet path.

BMP observes BGP state.



With router access, the **BGP Monitoring Protocol (BMP)** exports BGP state.
It is a collection protocol, not the public RIS dataset.

Read one public routing record.

NETHINKS (AS8319), Fulda. In a shell with curl and jq, such as operations01:

```
curl -fsS -G https://stat.ripe.net/data/routing-history/data.json \
  --data-urlencode resource=AS8319 \
  --data-urlencode starttime=2025-09-05T00:00:00 \
  --data-urlencode endtime=2025-09-07T00:00:00 \
  --data-urlencode include_first_hop=true \
  --data-urlencode min_peers=10 -o as8319.json

jq '.data.by_origin[0] | {origin, prefix: .prefixes[0].prefix,
  interval: .prefixes[0].timelines[0]}' as8319.json
```

Keep the full file. Inspect one record.

What did the collector actually see?

AS8319 example, checked 4 October 2026:

```
origin + first hop: 6939 8319
prefix:             149.218.0.0/17
interval, UTC:      5 Sep 00:00 to 7 Sep 07:59:59, 2025
full_peers_seeing: 11
```

One origin and a first hop. At least ten full-feed peers required.
Route visibility is not service availability.

Make the public measurement reproducible

Keep the details another person needs to repeat it:

- UTC time window
- unaffected comparison
- query or measurement IDs
- filters and thresholds

Save them with the conclusion.

Plan responsible measurement.

Prefer public data or your own lab.

For active probes, agree on **target, authorization, rate and stop condition**.

For collected data, define **access, retention and deletion**.
Collect only what the question needs.

Build an argument, then audit it.

Organize related work by the question the studies answer.

For each factual sentence, identify its role:

- our result → method and evidence
- prior result → original source
- protocol behavior → specification
- interpretation → inference and limitation

If you cannot classify it, check whether it belongs.

What does your project need next?

A sharper question? A useful comparison? Access to data?
An experiment that finally runs?

Use the project time to work on it. Bring questions as they come up.

Choose your group by 27 November, 23:59.

Your question, your next step

Write your **Research Brief**: question, closest prior work, baseline, available evidence and biggest feasibility risk.

Use today’s project time to check that you can access the evidence.

13 November: turn the question into a controlled experiment.

13 November 2026

Experimental design

Now the sampling question becomes a controlled comparison.

Assign treatments to runs.

Treatment: the sampling policy.
Outcome: detection and delay.
Experimental unit: one reset run assigned a treatment.

Keep fault workload, probe budget and reset conditions equal.

Ten samples from one run are not ten independent experiments.

Fix the comparison before running it

600-second runs with **60 probes per method**.

Method	One probe in each 10 s slot
Periodic	At the start
Randomized	Uniform random times with a recorded seed

Use the same fault schedule for each pair.
Compare detection fraction and delay among detections.

Specify the experiment boundary.

Use the smallest setting that can test the claim.

Record nodes and versions, link capacity and delay,
addresses and policy, probe locations and control path.

Keep control reachable when the workload fails.
Document any dependencies shared by control and experiment.

Two identical diagrams can conceal different experiments.

Virtual experiments have limits.

Virtual networks let us control conditions.

They do not reproduce hardware timing, optics or carrier scale by default.

Which part of your conclusion depends on the simulator?

Measure an independent reference.

Evidence	Tells us
Controller	Fault requested
Reference probe	Traffic affected
Evaluated detector	Fault reported

A **100 ms reference interval** limits onset-time resolution.
The evaluated detector cannot define its own ground truth.

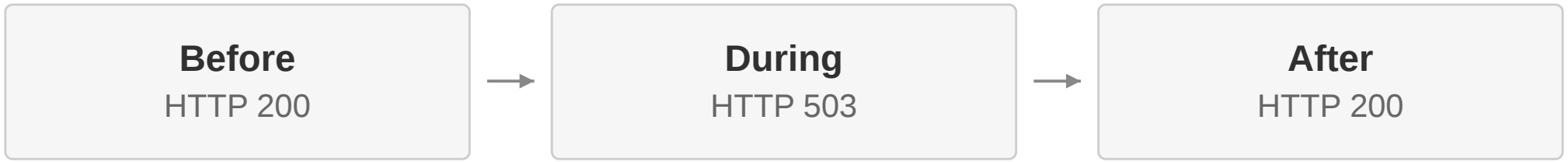
Predict the observations.

Customer IPv4 export to Lagoon is rejected. BGP stays up.

Compare	Look at
Lagoon vs Pacific	Customer advertisement
IPv4 vs IPv6	External reachability
Routing fault vs congestion	Rate and discards

What would contradict your diagnosis?

Control experiment: the endpoint fails



Same routes and interfaces. Different endpoint behavior.

An HTTP 503 response is not evidence of a routing failure.

Pair conditions, randomize order.

Replay each fault schedule for both methods at equal probe cost.
Reset between runs. Randomize which method runs first.

For jointly detected events: **difference = delay A − delay B**.
Report misses separately.

Block by host or workload and record contention.
Pairing does not remove clock error or unequal host load.

Startup or steady state?

Research question	Treatment of startup
How long until service recovers?	Startup is part of the outcome
What is steady-state overhead?	Apply a stated warm-up rule
Does cache warming help?	Compare defined cache conditions

Cold caches and convergence are real. Excluding them changes the experiment.

Decide how a run can fail

Outcome	Store
Detected	Flag + delay
Missed	False flag, no invented delay
Probe failed	Instrumentation failure
Reset failed	Invalid run + reason

Choose categories **before** comparing methods.

Turn the lab into an experiment



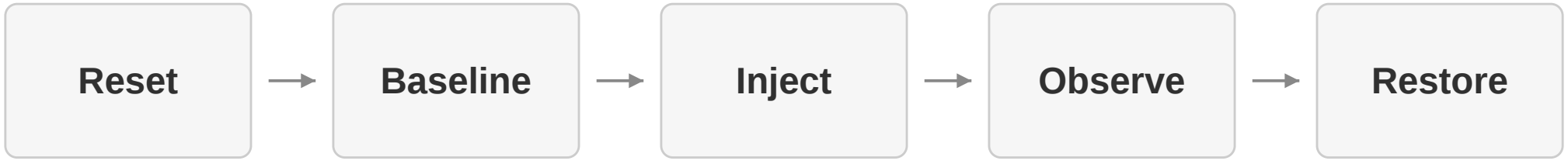
Pacific and IPv6 remain control observations.

Record the requested fault time and the first failed service check.
Keep measuring until the service is restored.

Config-change time is not traffic-failure time.

Define one experimental run

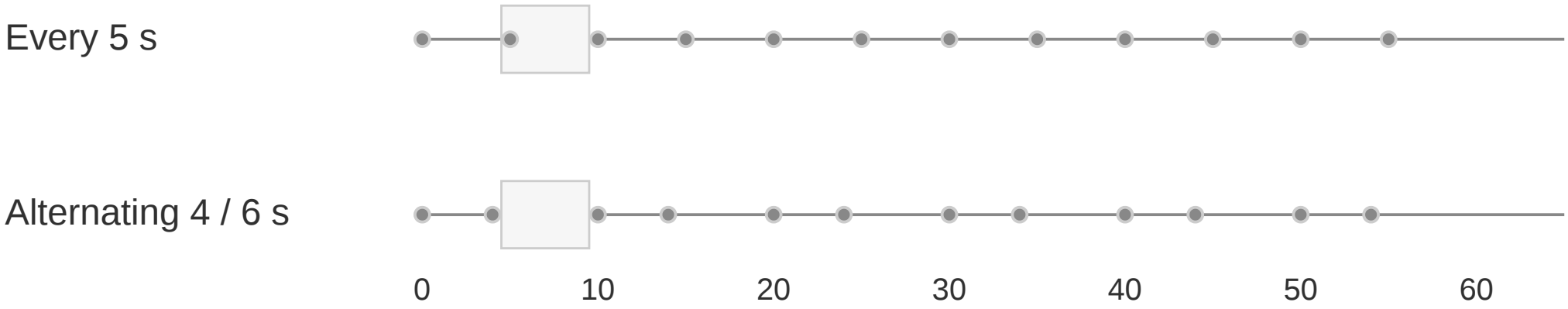
For the sampling comparison, repeat the same controlled sequence.



Run ID, seed, versions and raw results travel with the experiment.

A failed reset invalidates the comparison. Record it as a failed run.

Same budget. Different gaps.



Twelve probes per minute. A five-second fault can fit between probes.

Timing model, not the 10 s study: **5 s mean spacing**, 120 fault phases.

Compare detection and delay

Synthetic timing model: **120 phases, 5 s faults**.

Spacing	Detected	Missed	Median delay*
5 s	120	0	2.50 s
4 s / 6 s	108	12	2.25 s

Among detections. The lower median ignores 12 misses.

Task: compare detection delays

Five synthetic pairs. Delays in **milliseconds**.

Pair	1	2	3	4	5
A	11	12	10	13	54
B	17	18	16	17	18

Four minutes: which is usually faster, which more predictable?
Choose a summary and state its limit.

One outlier changes the mean

Statistic	A	B
Mean	20.0 ms	17.2 ms
Median	12 ms	17 ms
Range	10-54 ms	16-18 ms

A is earlier in 4/5 pairs. Its fifth run reverses the mean.

Small sample. This only illustrates the calculation.

More samples do not fix bias.

Source	Examine
Sampling uncertainty	Variation between runs
Measurement uncertainty	Clock and resolution
Bias	Unequal conditions
Instrumentation	Measurement overhead

More runs cannot repair a bad clock or unfair comparison.

Four types of validity

Type	Example failure
Construct	DNS used as HTTP health
Internal	Only one method runs under host load
External	One topology stands for all networks
Statistical	Correlated samples counted as independent

Task: review the experiment

Method	Host	Time
A	Idle	Morning
B	Shared and busy	Afternoon

Four minutes: identify the mixed effects.
Decide what to hold constant, randomize and record.

27 November 2026

Analysis and scientific argument

From paired runs to a defensible comparison.

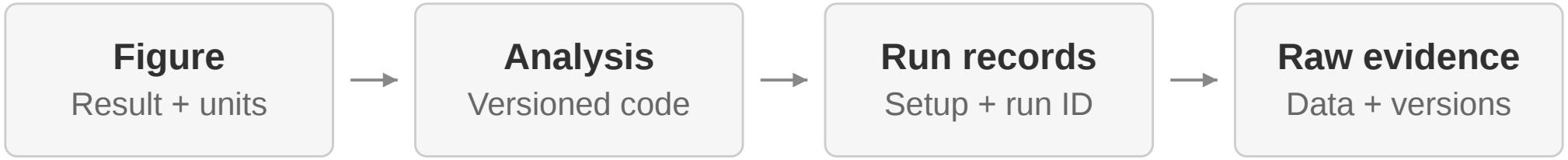
The graph is not the result. Check the evidence behind it.

Keep the mechanism visible.

Runs → detection counts → delays → uncertainty → claim

Inspect individual traces before aggregation.
Keep missed events in the detection denominator.

Follow one result back to its inputs



Follow one reported number all the way back.

Choose a number from your draft. Can another team reproduce it?

Keep one row per run

Illustrative paired runs. Delays in **milliseconds**.

Pair	Method	Detected	Delay
17	Periodic	Yes	820
17	Randomized	No	—
18	Periodic	Yes	240
18	Randomized	Yes	510

A miss stays in the detection denominator.

Missing, missed and timed out are different

Outcome	Meaning
timeout after 2 s	no completion observed by deadline
probe never ran	service outcome unknown
event not detected	miss for this detector
run excluded	rule + reason recorded

Dropping timeouts can make a broken service look fast.

A fast median can hide missed faults

First compare coverage. This separate example has **20 faults**.

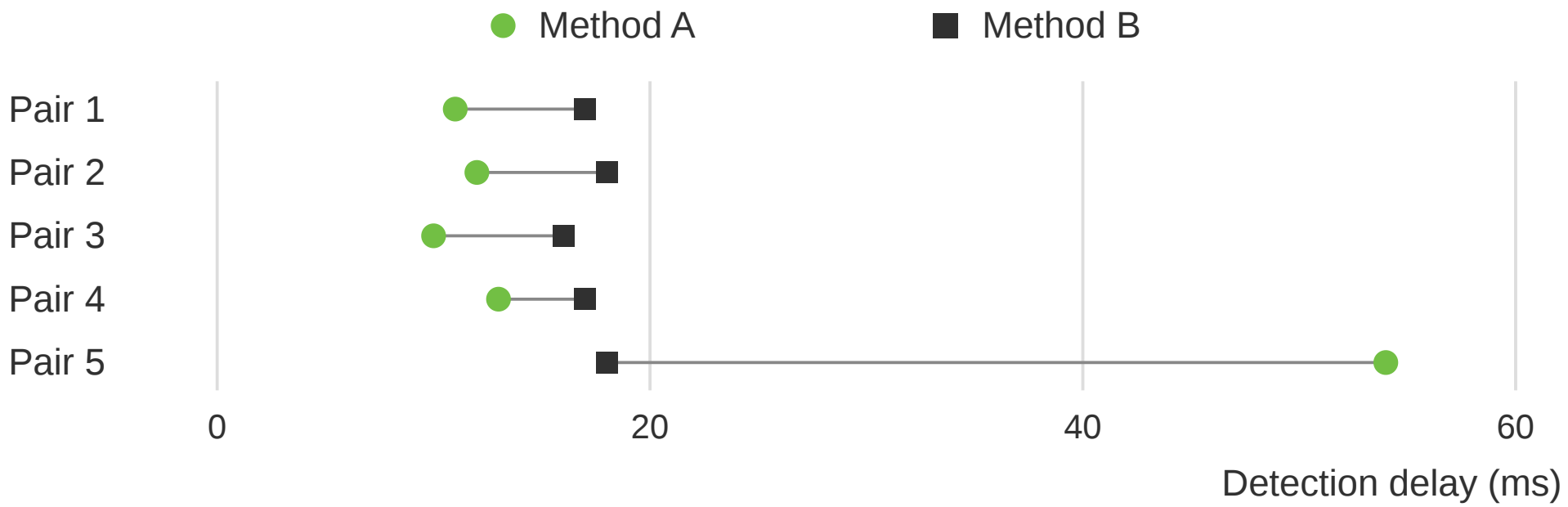
Method A: 18 of 20 detected
Median among detections: 1.0 s

Method B: 4 of 20 detected
Median among detections: 0.5 s

Synthetic example

Compare the same detected events before claiming that B is faster.

Compare within each pair.



Return to the five synthetic pairs from **13 November**, methods A and B.

Work with paired differences.

Pair	A − B, ms
1	−6
2	−6
3	−6
4	−4
5	+36

Mean +2.8 ms. Median −6 ms. A is earlier in four pairs.
Five synthetic pairs illustrate the calculation only.

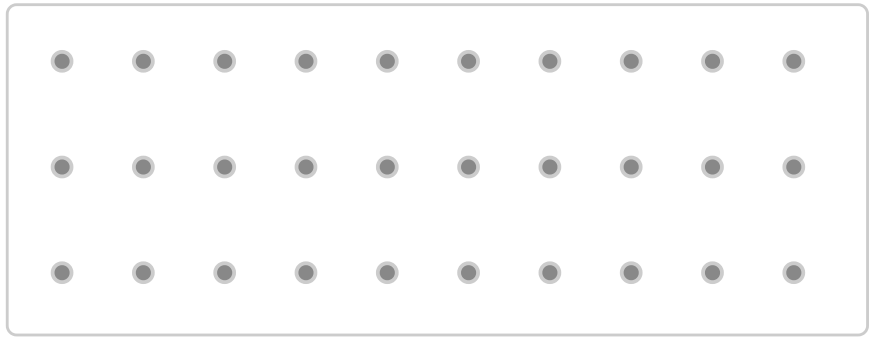
What are we estimating?

Term	Meaning in our study
Population	Runs under the stated conditions
Sample	Runs actually performed
Parameter	Population mean delay difference
Estimator	Observed mean paired difference

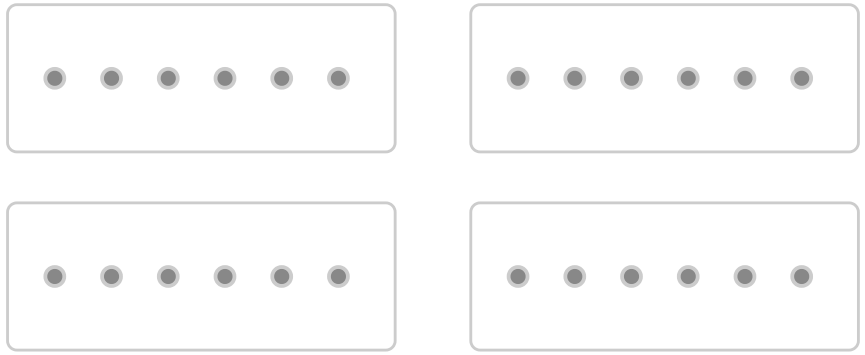
Compare delays only where **both methods detected the event**.

Independent observations

One run, many samples



Several independent runs



The unit of replication comes from the experimental design.

Use independent runs or matched pairs as the uncertainty unit.

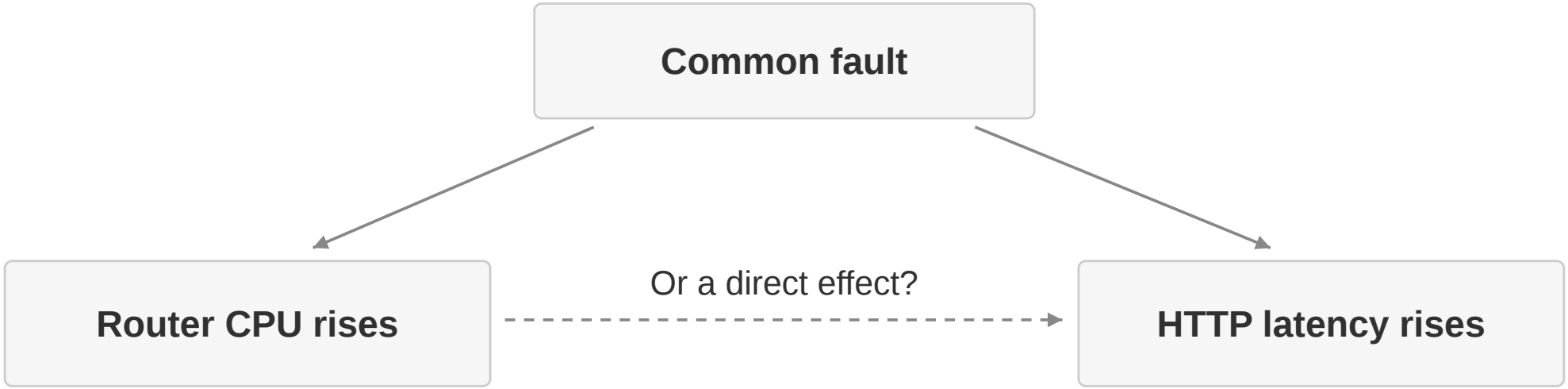
What does 95% coverage mean?

A 95% confidence-interval procedure covers the parameter in 95% of repeated datasets **when its assumptions hold**.

Bootstrap: repeatedly resample whole runs or matched pairs.
State the estimator, interval method and independence assumptions.

This is a property of the procedure, not a score for one interval.

Task: assess a causal claim



A shared cause can create correlation without a direct causal link.

Which observation or intervention could distinguish these explanations?

Is the effect useful?

1,000 ms → 990 ms: 10 ms saved, a 1% reduction.

Does it change a 30-second paging decision?
Could it matter to a sub-second control loop?

Report units, direction, uncertainty and failure rate.
A delay reduction is not a measured throughput gain.

Unexpected results

If the expected method does not win:

- check the implementation
- check the measurement
- repeat where needed
- explain the conditions
- report the result honestly

Keep the result, including when your preferred method loses.

Write from an evidence table.

Question	Evidence	Observation	Limit
RQ1	Figure 2	Lower median delay	Virtual lab
RQ2	Table I	More target load	One NOS

A results paragraph states **what changed, by how much and under which conditions**.

Practice reviewing before January

25 minutes, fictional study.

1. Your own review: 8 min
2. Compare in pairs: 7 min
3. Review together: 10 min

Use the same four criteria as the bonus review:

summary, method, strength/limitation, feasible revision.

A study for review

Fictional claim: **“Randomization halves delay.”**

	Periodic	**Randomized**
Host	Idle, morning	Loaded, afternoon
Budget	Equal	Equal
Detected	18 / 20	4 / 20
Median delay*	1.0 s	0.5 s

Among detections. What is confounded or omitted?

Review example

Review	Finding
Strength	Equal probe budget and reported misses
Problem	Host load changes with the method
Problem	Delays compare different detected subsets
Revision	Match conditions and report coverage beside delay

Explain how each revision makes the conclusion more defensible.

How is the course working for you?

Three minutes. Anonymous notes.

- What has helped your project?
- Where are you stuck?
- Pace: too slow / about right / too fast?
- One change that would help next time?

We discuss what I can still adjust.

11 December 2026

Forecasts, control loops and agents

We have compared detectors. Now test a forecast before it drives a controller.

What evidence would justify acting on the prediction?

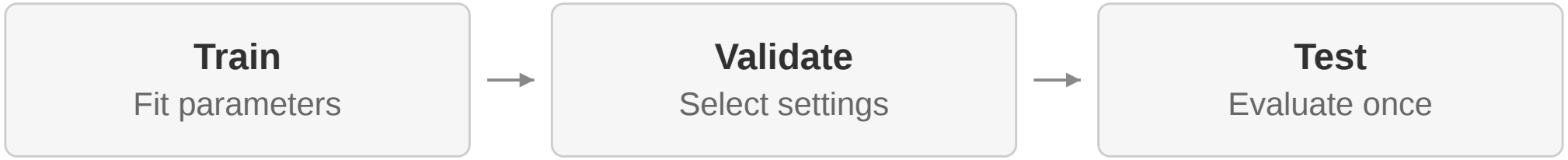
Can we warn before saturation?

Forecast synthetic traffic **one second ahead**.

Baseline	Challenger
Latest load value (persistence)	Ridge regression on past samples and periodic features

Same test data, horizon and missing-data rules.
Which model warns usefully?

Keep the future out of training.



Train on past data. Test on later data.

Use only information available at prediction time.

Traffic patterns change

Drift means the data or the predictive relationship changes.

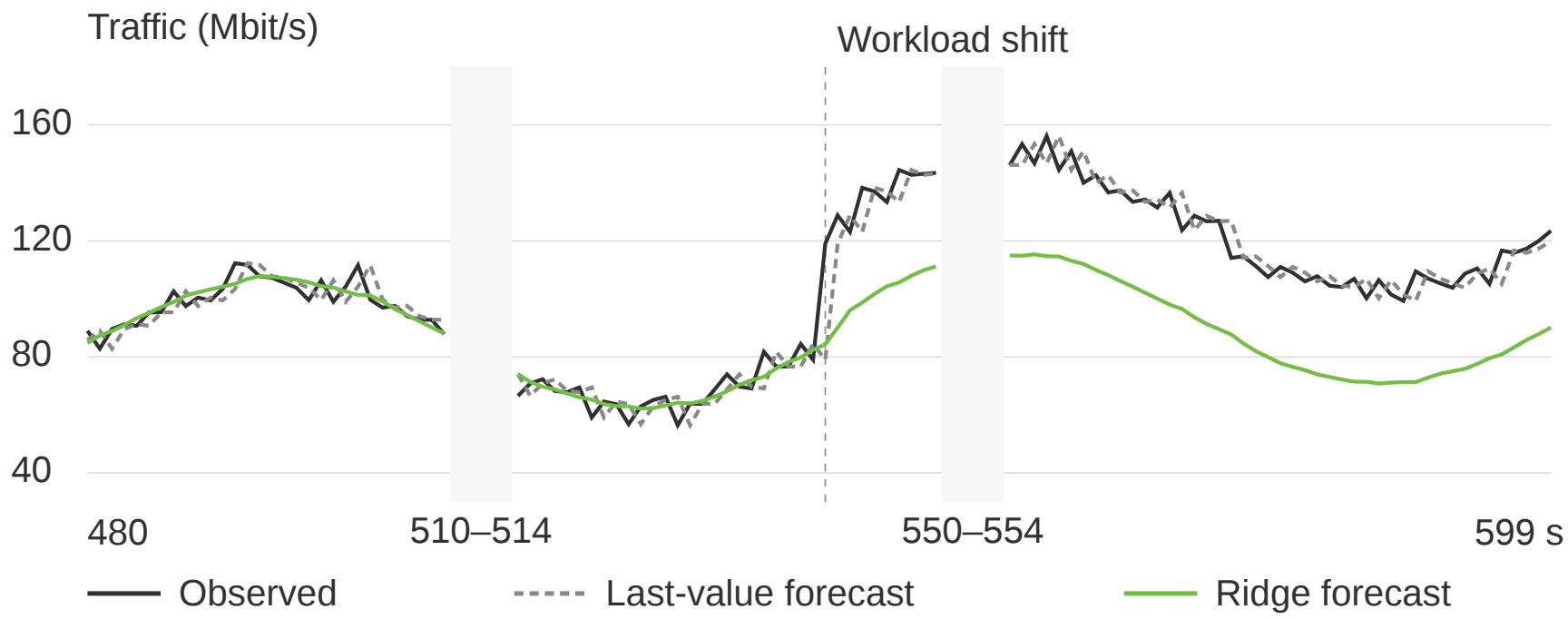
Network examples:

- route change moves traffic onto the link
- capacity upgrade changes utilization behavior

Watch prediction error.

Decide when to retrain, fall back, or stop automated action.

Forecasts miss the workload shift



Synthetic, 1 s ahead. **110 of 120 slots evaluated.** Shaded gaps excluded.

Score prediction and warning separately.

110 test slots. Warning threshold: 125 Mbit/s.

Predictor	MAE, Mbit/s	False alarms	Missed high-load slots
Persistence	4.74	3	3
Ridge	17.71	0	25

MAE = mean absolute error. No false alarms, but 25 misses.

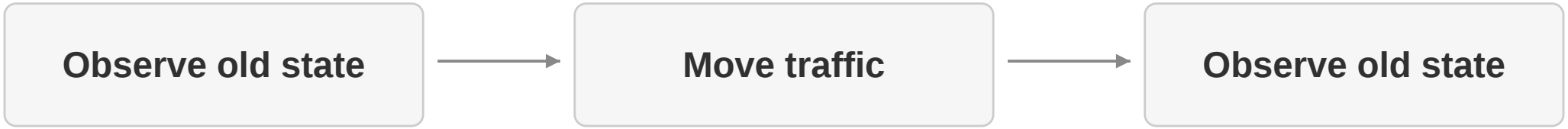
A controller cannot create capacity

Illustrative ReefNet load:

Transit	Capacity	Offered
Lagoon	50 Mbit/s	60 Mbit/s
Pacific	50 Mbit/s	30 Mbit/s

Move 10 Mbit/s: **50 / 40 Mbit/s**. Lagoon still has no headroom.
Would routing policy and traffic behavior permit the move?

Delayed feedback



Wait for fresh evidence before moving again.

Acting twice on the same stale sample can overshoot the target.

Put limits around the controller

Illustrative guardrails for the ReefNet controller:

```
allowed targets:  Lagoon, Pacific
maximum move:     10 Mbit/s per step
wait time:        30 s of fresh observations
stop if:          service degrades or data is stale
recovery:         previous allocation or manual handover
```

The same limits apply to rules, optimizers and LLMs.

Can an agent troubleshoot the network?

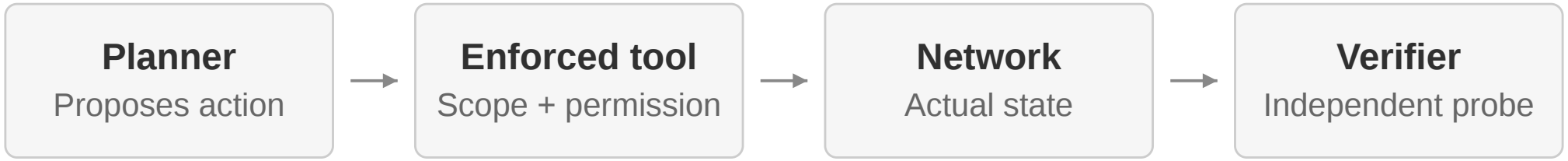
Give it a known fault, observations and bounded tools.

Compare with a deterministic runbook on the **same faults**.
Score the network state, not the explanation.

Enforce permissions and verification outside the planner.

The write boundary

The tool enforces authorization and execution.



Authorization stays outside the planner.

The verifier checks independently. Where do we enforce the one-device limit?

Limit what the agent can do

Example request for the BOB1 export-policy repair:

```
{
  "device": "edge01.bob1.reefnet.test",
  "bgp_group": "lagoon-v4",
  "expected_override": "BLOCK-CUSTOMER-V4",
  "action": "delete group export-policy",
  "dry_run": true
}
```

The tool enforces target scope, input checks, timeout and write permissions.

What should happen if the agent supplies another device or group?

Task: check the claimed repair

Fictional trace after write approval:

Step	Result
Read	`BLOCK-CUSTOMER-V4` attached
Delete	Command completed
External probe	Timeout
Agent reply	“The service is restored.”

One minute: how should the evaluator score this?

The explanation has recovered.

Recovery is still unproven.

Define safe success before scoring.

A repair succeeds only when:

1. the permitted target has the intended state
2. an independent service check passes
3. unrelated state is unchanged

Record refusal, failure, attempts, duration and cost.
Test stale state, uncertain writes and incorrect ticket assumptions.

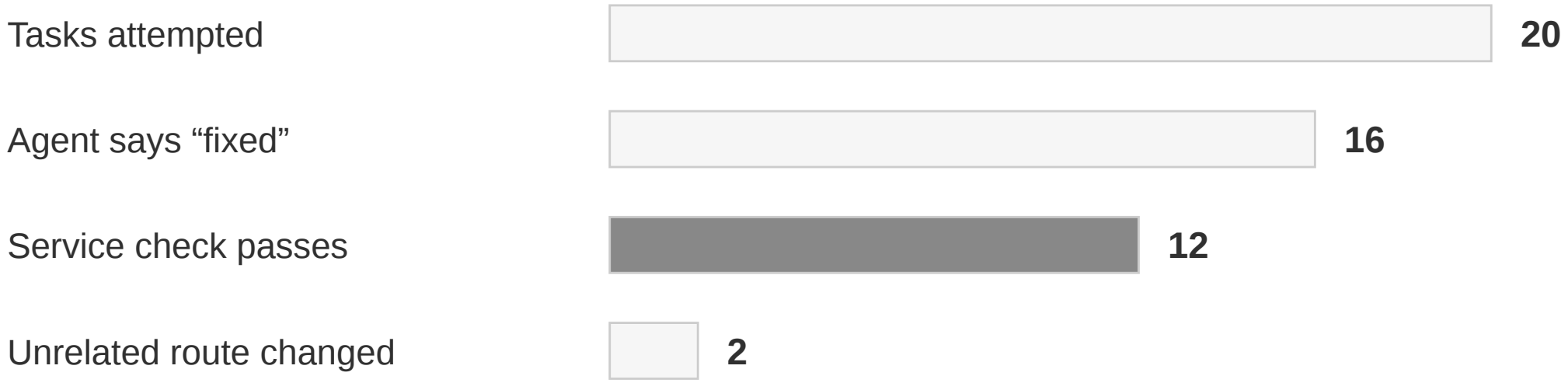
A benchmark needs known ground truth.

NIKA supplies reproducible troubleshooting scenarios and a tool interface.

Separate **detection**, **localization** and **root-cause identification**.
These are different tasks and need separate scores.

Read the paper's fault set and evaluation conditions before comparing agents.
A benchmark result is bounded by those conditions.

Task: assess the benchmark



Separate fictional repair benchmark: 20 attempts. Is 80% success justified?
What must we know about overlap and unintended changes?

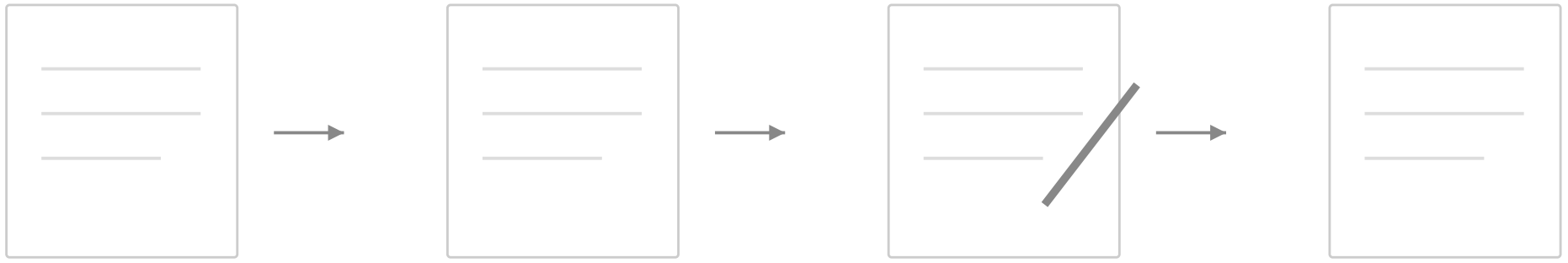
22 January 2027

In-class peer review

Read in class. Write **legibly by hand**.
German or English accepted.

Optional bonus review: **today only, no make-up date**.
Work independently, without AI or other aids.
Hand in today. The reviewed team receives a copy during this session.

Receive, read, write, return.



Receive
Paper + review sheet

Read
During the session

Write
By hand (DE / EN)

Return
To the instructor

Write your name and identify the reviewed paper.

The instructor keeps the original and makes a copy.
The reviewed team receives its copy **during this session**.

Review criteria: calibrate your judgment

Criterion	Enough	Not enough
Summary	Question and result	Topic only
Method	Choice assessed with evidence	"Unclear"
Judgment	Reasoned strength and limitation	Praise only
Revision	Feasible fix	"More experiments"

Apply the same criteria we announced in October.

Task: bound the conclusion

Two minutes. Reviewer discussion

> Our system makes backbone services reliable.

Five route changes, one emulated topology. Four restore the health check.

What can the authors claim? Which result would you ask them to add?

Detection delay and missed faults

Return to the fictional study from November.

Method	Detected faults	Median delay among detections
A	18 / 20	1.0 s
B	4 / 20	0.5 s

The draft calls B "the faster detector".

Write one review sentence. Name the hidden condition and ask for context.

Task: write a review comment

Two minutes. Fictional methods section

> We repeated each experiment several times and removed outliers.

Name the missing information and ask for a concrete correction.

A useful review explains how the issue affects the conclusion.

Task: rewrite the claim

> "Our method improves network reliability by 40%."

Evidence: mean detection delay for **one fault family**
in **one emulated topology**.

Draft a bounded claim. Mark the missing comparator and numbers.
Do not carry over **40%** without supporting results.

Name one experiment needed for a broader claim.

Now review the assigned paper

Work **independently**, without AI or other aids.
One page, legibly handwritten, in German or English.

- Summarize the question and result.
- Assess one methodological choice.
- Justify a strength and a limitation.
- Propose a feasible revision.

Use the review to revise your paper

For each comment: **clarify, add evidence, fix analysis or narrow the claim**.

> We reran both methods at equal budget, added Figure 3
> and narrowed the conclusion.

If you disagree, explain why and point to the result.

5 February 2027

Presentation and reproducibility

Show the result. Defend the choices. Let someone else reproduce the evidence.

Presentation and reproduction

8 minutes presentation + 7 minutes questions

Present the problem, comparison, main result and limitation.

Then trace one result back to its data and regenerate it.

The presentation itself has no grade weight. Use it as the last technical check.

Test the artifact handover.

Another team opens the repository.

Can they find:

- exact version used in the paper
- required inputs and access
- command for the main figure
- expected output
- known limits

"Works on my laptop" means the laptop is still an undocumented dependency.

Task: complete the instructions

Two minutes. Incomplete README

> Install the dependencies. Start the network.
> Run the experiment. Plot the results.

Choose three missing details that would stop another team.

Give concrete replacements, including how they would recognize success.

Publication-quality figures

Check the figure at its **final paper size**.

- Readable labels, explicit units
- Traceable data, consistent terms
- Distinguishable in grayscale
- No decoration without meaning

Export the plot itself, not a dashboard screenshot.

Before submitting

- Claims match evidence and references.
- Figures trace back to data.
- Failures and limitations remain visible.
- Required AI use is disclosed.
- Every author has read the paper.

Paper and artifact describe the same study.

Submission package

- **PDF:** opens correctly, six pages including references, declaration excluded, correct authors
- **Artifact:** accessible to the examiner, tagged version matches the paper
- **Declaration:** individual contributions are complete
- **AI use:** required disclosure is in the Acknowledgments

AI disclosure

OpenAI Codex assisted with this slide deck.

Assistance covered structure, wording, diagrams, teaching examples and checks of formatting and rendered output.

Thanks

Network Management and Monitoring

Questions, results or a packet capture worth discussing?